



NANYANG TECHNOLOGICAL UNIVERSITY

**CCDS25-0642: Trustworthiness of Large
Language Models**

A thesis submitted to the Nanyang Technological University in
partial fulfilment of the requirement for the degree of Bachelor of
Computing in Computer Science

by

Gu Shucheng

Project Supervisor: Associate Professor Luu Anh Tuan
College of Computing and Data Science
Academic Year 2025/2026

Abstract

This thesis investigates hallucination mitigation in question answering grounded in given knowledge and formulates the problem as one of faithfulness to the provided knowledge: when the knowledge is sufficient to support an answer, the model should answer based on that knowledge; when it is insufficient to support a verifiable answer, the model should refuse rather than guess. To address this objective, the thesis proposes a protocol-based supervised fine-tuning (SFT) approach that explicitly models answer behavior as a set of interdependent structured subtasks, including answerability judgment, evidence extraction, minimal rationale generation, short-answer generation, and discrete confidence self-assessment. On this basis, it constructs a complete workflow spanning data construction, teacher-supervision generation, sample filtering, SFT training, and unified evaluation.

In the experiments, this thesis constructs answerable and derived unanswerable instances from the HotpotQA distractor subset, uses a strong teacher model to produce protocolized supervision, and applies LoRA-based SFT to Qwen3-4B. The best SFT checkpoint achieves 88.2%, 94.6%, and 92.6% on answer correctness, answerability accuracy, and semantic faithfulness, respectively, representing gains of 9.2, 7.2, and 11.6 percentage points over unconstrained natural-language answering by the base model. Further analyses show that these gains arise not primarily from surface formatting, but from a more stable refusal boundary, stronger evidence copying behavior, and more reliable evidence-grounded reasoning. Comparisons with auxiliary baselines further indicate that a protocol internalized through SFT yields substantial gains over obtaining protocol-structured outputs through detailed prompting and example demonstrations alone, and also offers a modest advantage over generic thinking on this task.

These results suggest that, for QA grounded in given knowledge, protocolizing answer behavior and internalizing high-quality structured supervision into model parameters can reduce unsupported answering and improve behavioral verifiability.

Acknowledgements

I would like to express my sincere gratitude to my final year project supervisor, Associate Professor Luu Anh Tuan, for his invaluable guidance, support, and assistance throughout this project. His insightful advice and encouragement have been instrumental in the completion of this work.

I would also like to extend my heartfelt appreciation to Nanyang Technological University for providing me with valuable learning opportunities and the resources necessary for my academic and personal development.

Finally, I am deeply grateful to my family for their unwavering support, encouragement, and understanding throughout my studies and during the completion of this project.

Table of Contents

1 Introduction	9
1.1 Background	9
1.2 Scope and Objective	10
1.2.1 Scope.....	10
1.2.2 Objective.....	11
2 Related Concepts and Methods.....	12
2.1 Definitions and Taxonomy of Hallucination.....	12
2.1.1 Definition of Hallucination	12
2.1.2 Intrinsic vs. Extrinsic.....	12
2.1.3 Faithfulness vs. Factuality	12
2.1.4 Definition Adopted in This Thesis	13
2.1.5 Related Neighboring Concepts	13
2.2 Detection Methods for Faithfulness Hallucination	14
2.2.1 Manual Annotation: The Gold Standard but Costly	14
2.2.2 NLI-Based Detection	14
2.2.3 QA-Based / Information-Extraction-Based Detection.....	15
2.2.4 LLM-as-Judge.....	15
2.3 Post-Training with SFT	16
2.3.1 Basic Principles of SFT	16
2.3.2 Suitability of SFT	16
2.3.3 Limitations of Prompt Engineering for Protocol Compliance	17
2.3.4 Rationale for Prioritizing SFT over DPO and RLHF	18
3 Methodology	19
3.1 Task Formulation.....	19
3.1.1 Natural-Language Formulation of the Task.....	21
3.1.2 Answerable and Unanswerable Cases	22
3.2 Structured Output Protocol.....	22
3.2.1 Formal Protocol.....	22
3.2.2 Semantics and Functional Justification of the Fields	22
3.3 Validation, Checking, and Calibration Framework	24
3.3.1 Overview of the Multi-Stage Checking Framework.....	25
3.3.2 Calibration Procedures.....	29
3.4 Data Construction Strategy	30
3.4.1 Construction of Answerable Instances.....	30
3.4.2 Deriving Unanswerable Instances from Answerable Ones	31
3.4.3 Prompt-Length Filtering Before Teacher Generation	31
3.4.4 Answerability Construction Ratio	31
3.4.5 Independent Stable Split Assignment.....	32
3.5 Teacher Supervision Generation and Selection	32
3.5.1 Teacher Prompt and Infer Prompt.....	32
3.5.2 Teacher Generation Strategy.....	32
3.5.3 Basic Validity Checks.....	33

3.5.4 Validation and Selection for Answerable Candidates.....	33
3.5.5 Validation and Selection for Unanswerable Candidates.....	33
3.5.6 Post-hoc Confidence Relabeling.....	34
3.6 Student Model Training.....	34
3.6.1 Training Data Construction.....	34
3.6.2 SFT Trainer.....	35
3.6.3 LoRA with PEFT.....	35
3.7 Evaluation Method and Baseline Setup.....	35
3.7.1 Validation-Stage Checkpoint Evaluation	35
3.7.2 Test-Stage Comparative Evaluation and Baselines	36
3.7.3 Unified Evaluation Path for Protocol and Flat Outputs.....	37
4 Experimental Setup.....	39
4.1 Dataset and Split.....	39
4.1.1 Dataset Choice.....	39
4.1.2 Construction Parameters.....	39
4.1.3 Train / Validation / Test Split.....	40
4.2 Prompt Templates	40
4.2.1 Prompt Families Used in the Experiments	40
4.3 Models and Inference Configurations.....	42
4.3.1 Qwen3-4B	42
4.3.2 Teacher and Auxiliary Models	43
4.3.3 Inference Configuration Table	43
4.4 Training Configuration	44
4.4.1 SFT Hyperparameters.....	44
4.4.2 LoRA Configuration.....	45
4.5 Checker Parameters and Calibration	45
4.5.1 Tokenizer-Length Check.....	45
4.5.2 Parse, Protocol, Answerability, and Evidence Substring Checks	45
4.5.3 Correctness Checks	46
4.5.4 Semantic Checks.....	47
4.5.5 DeepEval Hallucination Check.....	47
4.6 Metric Set and Checkpoint Selection	47
4.6.1 Primary End-to-End Metrics	48
4.6.2 Secondary Diagnostic Metrics.....	49
4.6.3 Checkpoint Selection Rule	50
5 Results and Analysis	52
5.1 Validation Results and Checkpoint Selection.....	52
5.1.1 Validation-Stage Trend Analysis and Checkpoint Selection.....	52
5.2 Main Test Results	54
5.3 Secondary Analyses.....	56
5.3.1 Mechanism Decomposition.....	56
5.3.2 Answerable vs. Unanswerable.....	57
5.3.3 Confidence Effectiveness	59
5.4 Case Studies.....	61
5.4.1 Success Case: SFT Corrects a Relation-Direction Error on an Answerable Question	61

5.4.2 Failure Case I: Answering an Unanswerable League Question from Incomplete Team Information.....	62
5.4.3 Failure Case II: Entering the Answerable Path but Drifting Away from the Question's Semantic Criterion.....	64
5.4.4 Summary of the Case Studies	65
5.5 Summary and Discussion.....	65
5.6 Threats to Validity	67
5.6.1 Internal validity	67
5.6.2 Construct validity.....	67
5.6.3 External validity	68
5.6.4 Statistical conclusion validity.....	68
6 Conclusion and Future Work.....	69
6.1 Conclusion.....	69
6.2 Main Contributions.....	70
6.3 Future Work.....	70
Appendices	72
References	76

List of Tables

Table 1 Source-stage data construction parameters	39
Table 2 Split-specific sampling caps	40
Table 3 Inference configurations	43
Table 4 SFT hyperparameters.....	44
Table 5 LoRA configuration.....	45
Table 6 Answer equivalence matcher calibration results	46
Table 7 Semantic NLI calibration results	47
Table 8 Key validation metrics and checkpoint selection results	52
Table 9 Main results on the test set	54
Table 10 Structure- and mechanism-related metrics	56
Table 11 Answerable / unanswerable stratified results	57
Table 12 Confidence results for structured methods.....	59
Table 13 Confidence Resolution of structured methods	60

List of Figures

Figure 1 Workflow of data construction	20
Figure 2 Workflow of training and evaluation	21
Figure 3 Validation curves.....	52
Figure 4 Main test metrics	55
Figure 5 Stratified test metrics.....	58
Figure 6 Confidence-bucket success rates.....	60

1 Introduction

1.1 Background

In recent years, large language models (LLMs) have advanced rapidly. With continued progress in model scale, training-data scale, and training methodology, LLMs have demonstrated remarkable capabilities across a wide range of tasks, including language understanding, text generation, question answering, coding assistance, educational support, office automation, and content analysis ([1], [2]).

This broad application potential stems from several distinctive strengths of LLMs. Unlike traditional models designed for specific task, LLMs can address diverse problems through a unified natural-language interface and often exhibit strong generalization even in the absence of extensive task-specific engineering ([1]).

However, these advances also bring potential risks. In serious applications that demand factual accuracy, consistency with evidence, and accountability of outputs, models often generate content that appears fluent, complete, and even persuasive, but is in fact unsupported, inconsistent with the input evidence, or unverifiable. This phenomenon is commonly referred to as hallucination. Unlike errors in traditional systems, which are often explicitly exposed, LLM hallucinations frequently appear in natural, coherent, and linguistically polished form, making them harder for non-expert users to detect in time. In open-ended casual interaction, such errors may amount to little more than minor flaws; in high-stakes settings such as enterprise QA, knowledge retrieval, medical assistance, legal support, financial analysis, and education, however, hallucinations directly weaken system trustworthiness and significantly increase both usage cost and deployment risk ([3], [4]).

For this reason, hallucination has become one of the major obstacles limiting the deployment of LLMs in larger-scale and higher-value scenarios. Even when a system is highly capable, if it frequently produces unsupported answers, users must continually verify its outputs externally and may default to distrusting them. Under such conditions, the efficiency gains promised by the model are offset by verification costs. Reducing

hallucination is therefore not merely a matter of improving a local metric; it is a crucial step toward broader real-world adoption of LLMs ([3], [4]).

Existing work has explored many directions to mitigate this problem, including prompt design, external retrieval, decoding control, post-training optimization, and factuality checking, and important progress has been made. These methods reduce the risk of evidence drift at different stages of the answering process. Overall, however, reliability still lacks a stable, unified, and operational guarantee mechanism. On the one hand, different types of unreliable generation arise from different causes and therefore often require targeted remedies. On the other hand, natural-language generation itself lacks explicit and rule-governed intermediate structure, which makes error detection, localization, handling, and coordination across intervention methods difficult. Against this background, this thesis aims to develop a systematic approach spanning training, generation, and validation so that LLMs can answer more reliably and more verifiably under evidence constraints.

1.2 Scope and Objective

1.2.1 Scope

This thesis restricts hallucination mitigation to a reliability problem under bounded evidence. Specifically, given knowledge K and a question Q , when K entails or sufficiently supports an answer, the model should answer on the basis of K ; when K is insufficient to support an answer, the model should refuse rather than guess. Accordingly, hallucination in this thesis refers specifically to generated content that lacks support with respect to the given knowledge, rather than to inconsistency with open-domain real-world knowledge.

This restriction is motivated by two considerations. First, constraining correctness to the scope of the provided knowledge makes the source of truth explicit and avoids conflating multiple failure modes, such as retrieval failure, missing evidence, outdated external knowledge, and bias in parametric memory. This makes the object of study more focused and makes both human review and automatic evaluation more operational and verifiable. Second, in applications such as retrieval-augmented generation, enterprise QA, and document-driven assistants, the system usually does not need to answer freely in the open

domain; instead, it needs to use the given evidence faithfully. In such settings, an unsupported but superficially plausible answer is often riskier than an explicit refusal. Studying constrained answer behavior grounded in given knowledge is therefore of greater practical significance ([5], [6], [4]).

1.2.2 Objective

Within the scope defined above, this thesis argues that the answer generation process should be modeled as a set of interdependent constrained subtasks, including answerability judgment, evidence extraction, minimal reasoning, answer generation, and reliability self-assessment. Doing so turns an originally implicit and mixed generation behavior into a structured decision process that can be supervised, verified, and calibrated. This protocol is valuable for at least three reasons.

First, it defines an ordered decision process that encourages the model to learn a controlled behavior pattern of judging first, gathering evidence second, and answering last.

Second, it decomposes what would otherwise be a mixed and uncertain generation process into a fixed sequence of decision stages, allowing each part to be evaluated and optimized separately and making it easier to identify weak links in model behavior.

Third, compared with unconstrained natural-language generation, checking each constrained component is more reliable and less expensive. This is useful both for development-time optimization and evaluation and for post-generation control in practice.

On the basis of this protocol, the thesis develops a complete methodological framework centered on constrained answer behavior. The framework consists of three tightly connected parts: protocol definition, protocol-based SFT data construction and training, and protocol-based evaluation.

2 Related Concepts and Methods

2.1 Definitions and Taxonomy of Hallucination

2.1.1 Definition of Hallucination

In natural-language generation research, hallucination usually refers to model outputs that are fluent, semantically coherent, and seemingly plausible on the surface, but inconsistent with facts, input context, or verifiable evidence. Such outputs are dangerous precisely because their lack of grounding is often masked by high readability and naturalness ([3], [4]).

Hallucination is not a single, stable, and task-invariant concept. Different research traditions emphasize different reference frames: some judge hallucination relative to a given source text, some relative to open-domain real-world facts, and others include deviations from instructions or internal logical inconsistency. Therefore, before conducting a concrete study, one must first clarify what the output is expected to be faithful to. In this thesis, that reference frame is not open-world factuality, but the given knowledge. Accordingly, the hallucination discussed here is essentially an evidence-grounded faithfulness problem rather than an open-domain factuality problem ([5], [4]).

2.1.2 Intrinsic vs. Extrinsic

In research on grounded generation and summarization, hallucination is commonly divided into two categories:

- Intrinsic hallucination: the output directly conflicts with the designated source, meaning that the source text contains explicit information inconsistent with the output.
- Extrinsic hallucination: the output cannot be supported by the designated source, meaning that it is not entailed by the source, though it may not be explicitly contradicted by it.

([5])

2.1.3 Faithfulness vs. Factuality

More recent LLM literature tends to classify hallucination according to which consistency constraint is violated. Huang et al. summarize hallucination into two broad categories:

- Faithfulness hallucination: the output is inconsistent with the user input, the provided context, or the model's own internal logic.
- Factuality hallucination: the output is inconsistent with verifiable real-world facts.

([4])

Conceptually, the outputs of interest in this thesis, namely answers that are unsupported by or conflict with the given knowledge, form a specific subtype of faithfulness hallucination. The intrinsic / extrinsic distinction can in turn be viewed as a further decomposition within faithfulness hallucination.

2.1.4 Definition Adopted in This Thesis

This thesis is primarily concerned with faithfulness hallucination. More specifically: Given knowledge K and a question Q , if a model output A contains a proposition c , and c cannot be supported by K or conflicts with K , then the output is considered hallucinatory. The thesis does not investigate whether A is consistent with open-domain world knowledge; it only asks whether A is faithful to the given knowledge. Under this definition:

- If $K \models c$, then c is supported.
- If $K \models \neg c$, then c is contradicted.
- If $K \not\models c$ and $K \not\models \neg c$, then c is unsupported.

The latter two cases are both treated as hallucination in this thesis. The first case corresponds to verifiable answering, while the latter two correspond to unreliable answering ([5], [6]).

2.1.5 Related Neighboring Concepts

To avoid terminological confusion, this thesis further distinguishes several related concepts.

1. Faithfulness / groundedness / attribution.

These terms are similar in many studies and all emphasize that the output should be supportable by a given evidence text. This thesis mainly uses faithfulness because it

maps most directly onto the constraint of being faithful to designated knowledge. In the evaluation literature, however, groundedness, attributable generation, and factual consistency with respect to source may be treated as near-synonymous expressions ([5], [6]).

2. Refusal / abstention.

When K is insufficient to support an answer, an output such as "I cannot determine the answer from the provided knowledge" does not constitute hallucination. On the contrary, it is the correct behavior under the protocol proposed in this thesis. In this setting, refusal is therefore not an error category, but part of the task definition itself.

2.2 Detection Methods for Faithfulness Hallucination

2.2.1 Manual Annotation: The Gold Standard but Costly

Manual annotation remains the gold standard for natural-language evaluation tasks. Its main drawbacks, however, are high cost, slow turnaround, and difficulty in controlling annotator consistency. In this thesis, manual annotation is treated as a validation and calibration mechanism for key automatic checkers rather than as the primary method for large-scale data labeling or data generation.

2.2.2 NLI-Based Detection

Natural language inference (NLI) models faithfulness detection as follows: given an evidence text as the premise and a candidate output, or a proposition extracted from it, as the hypothesis, the model predicts whether the relation between the two is one of:

- entailment / support: the evidence supports the output;
- neutral / unsupported: the evidence cannot support the output;
- contradiction: the evidence conflicts with the output.

This framework directly matches the supported-unsupported-contradicted trichotomy used in this thesis and is therefore highly compatible with the task ([7], [6]). It is also relatively inexpensive and suitable for fast automatic judgment at scale, making it useful for data filtering, pre-training cleanup, and intermediate validation.

The main limitations of NLI-based judgment are as follows.

It typically operates on sentence pairs, making it less suitable for long texts and multi-hop tasks. In practice, sentence splitting, chunking, or aggregation is often required; otherwise, local judgments may miss cross-sentence dependencies ([6]). In this thesis, the output protocol constrains the structure of the rationale and answer, making them more amenable to NLI-based checking.

In addition, NLI models and their training data are designed primarily for truth-related semantic judgment and are not well suited to judging whether a refusal is appropriate. Accordingly, this thesis handles refusal cases using other fields and rules and applies NLI only to non-refusal outputs.

2.2.3 QA-Based / Information-Extraction-Based Detection

Another major line of work is QA-based evaluation. The core idea is to first extract information units from the output text, or automatically generate questions around those information units; then answer those questions using the source document; and finally compare the source-derived answers with the answers implied by the output text in order to measure whether the output is faithful to the source ([8], [9]). Because the underlying signal differs from that of NLI, QA-based evaluation is complementary to NLI-based evaluation ([10]).

Even so, this thesis does not adopt QA-based or IE-based methods as its primary detection mechanism. The reason is that the task studied here is not global factual verification for long-form summarization or long-form generation, and the dataset already provides questions together with corresponding reference answers. Therefore, answer-reference agreement is directly used as one of the main evaluation signals, with implementation details described later in the Methodology chapter.

2.2.4 LLM-as-Judge

With the rise of instruction-following LLMs, an increasing body of work has begun to use LLMs themselves as evaluators, namely LLM-as-judge. Such methods usually design an evaluation prompt so that the LLM can score, classify, or explain an output according to a given standard. This has become an important branch of faithfulness detection. Its key advantage is that it does not require training a dedicated classifier and can transfer more easily to new tasks that lack annotated data ([4]).

This thesis regards LLM-as-judge as a supplement to human evaluation rather than the sole or final standard of trustworthiness. It is especially suitable for the following roles:

- assisting human review;
- providing additional explanations for boundary cases;
- supplying exploratory signals when annotated data are unavailable.

2.3 Post-Training with SFT

2.3.1 Basic Principles of SFT

Supervised fine-tuning (SFT) refers to continuing supervised learning on top of a pretrained language model using labeled input-output pairs so that the model adapts to a specific task, output style, or behavioral constraint ([2]). In the autoregressive language-model setting, the standard training objective of SFT is to maximize conditional likelihood, or equivalently to minimize negative log-likelihood:

$$\max_{\theta} \sum_{(x,y) \in \mathcal{D}} \log p_{\theta}(y | x)$$

where x denotes the input, y the target output, and θ the model parameters. The meaning of this objective is straightforward: given an input, the model is encouraged to generate the target behavior demonstrated in the training data with higher probability.

2.3.2 Suitability of SFT

The goal of this thesis is to teach the model a constrained answering protocol. This protocol includes:

- a fixed output structure and fixed fields;
- dependency relations among the fields;
- conditions for answerability judgment;
- boundaries for evidence extraction;
- a minimal reasoning pattern from evidence to answer;
- trigger rules for self-assessment and refusal.

SFT is particularly well suited to this setting. It uses input-standardized-output pairs to demonstrate exactly how the protocol should be executed, thereby converting previously implicit and drift-prone generation habits into an output procedure that can be imitated and reproduced stably. As long as the training samples consistently instantiate the

protocol, SFT can directly compress these behavior patterns into the model parameters, allowing them to be reproduced at inference time with relatively little prompting overhead.

2.3.3 Limitations of Prompt Engineering for Protocol Compliance

Prompt engineering is an important component of this thesis, but it cannot fully replace SFT for three main reasons.

First, LLMs can indeed perform novel tasks through instructions and few-shot examples without explicit task-specific training ([1], [11]). However, the objective of this thesis is not merely to make the model output a certain format. The model must also learn whether it should answer, which spans constitute key evidence, how to avoid mixing in parametric world knowledge beyond the given knowledge, when it should refuse, and how to perform reliable self-assessment. These requirements include not only rules that can be stated explicitly, but also many implicit competencies that cannot be exhaustively enumerated through prompting alone. A model may understand what is being asked, yet still fail to do it reliably.

Second, one of the central aims of the thesis is to improve the reliability of a relatively small model. SFT can convert behaviors that would otherwise need to be imposed through a complex prompt at inference time into parameterized habits learned in advance. This allows more stable behavior with lighter execution prompts at deployment time, reducing inference cost and avoiding overly long or complex prompts becoming a burden in their own right.

Accordingly, this thesis treats prompt engineering as a necessary supporting method that works together with SFT. During data construction, detailed prompts are used to call strong LLMs and filter the resulting outputs to obtain protocol-compliant training samples. During training and evaluation, shorter prompts that still contain the essential constraints are used to trigger generation.

The thesis also uses stronger prompts together with few-shot examples to query an untrained model and form comparative baselines. In the formal evaluation, however, all compared pathways use single-pass deterministic decoding so that the results are more reproducible and comparable, enabling a fair comparison between prompt-only methods and protocol-based SFT.

2.3.4 Rationale for Prioritizing SFT over DPO and RLHF

In modern LLM post-training, it is common to use SFT first to teach the model the basic task format and target behavior, and then apply preference-learning methods to further improve output quality ([2]).

For the task considered here, however, the primary goal is to learn a standardized output behavior through learning from demonstrations. Once the protocol behavior is stable, preference-training methods such as DPO or RLHF may serve as a next-stage extension to further optimize aspects such as the naturalness of refusal wording, the calibration of self-reported confidence, evidence-selection preference, or multi-objective trade-offs. Those questions lie outside the scope of this thesis.

3 Methodology

3.1 Task Formulation

The core method proposed in this thesis is a pipeline consisting of data construction and annotation preparation, teacher-supervision generation and filtering, protocol-based SFT training, and unified checking and evaluation.

Starting from raw multi-hop QA data, the thesis first constructs training, validation, and test data containing both answerable instances and derived unanswerable instances, and represents each sample uniformly as a knowledge + question input. A strong teacher model then generates candidate completions under the structured output protocol defined in this thesis. These candidates are filtered through multiple stages, including parse checking, protocol checking, answerability-consistency checking, evidence-substring checking, and correctness / semantic checking, so that only high-quality supervision is retained for student-model training.

During training, the student model learns to output a structured result containing answerability, evidence, rationale, answer, and confidence. In this way, the behavior pattern of first deciding whether the question is answerable, then selecting evidence, and finally performing minimal reasoning before answering is internalized as a parameterized capability. After training, the thesis compares SFT checkpoints with multiple baselines under a unified evaluation framework and evaluates them from the perspectives of correctness, answerability, evidence faithfulness, and semantic support using both rule-based and model-based checkers.

The overall workflow is visualized below.

Figure 1 Workflow of data construction

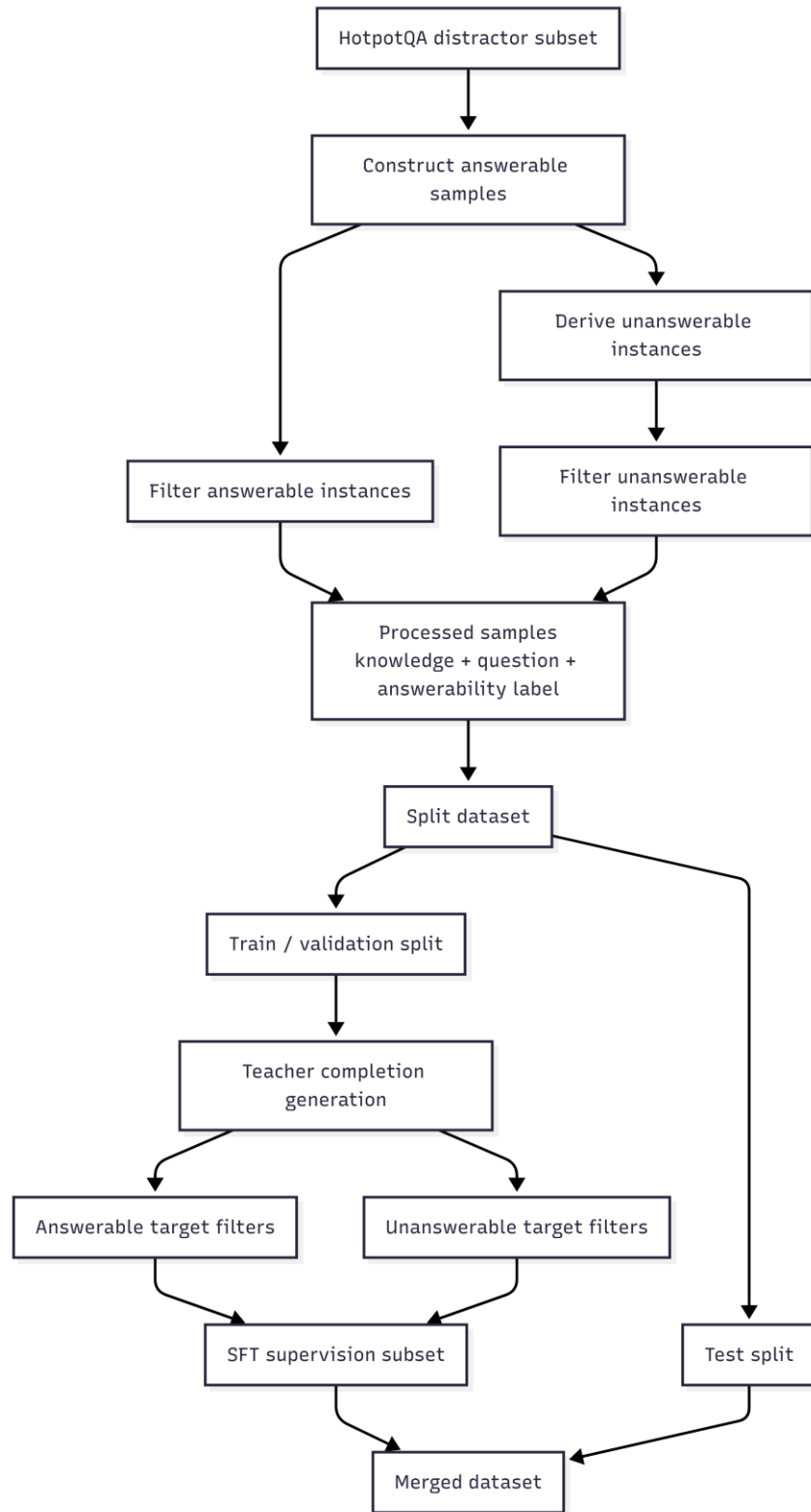
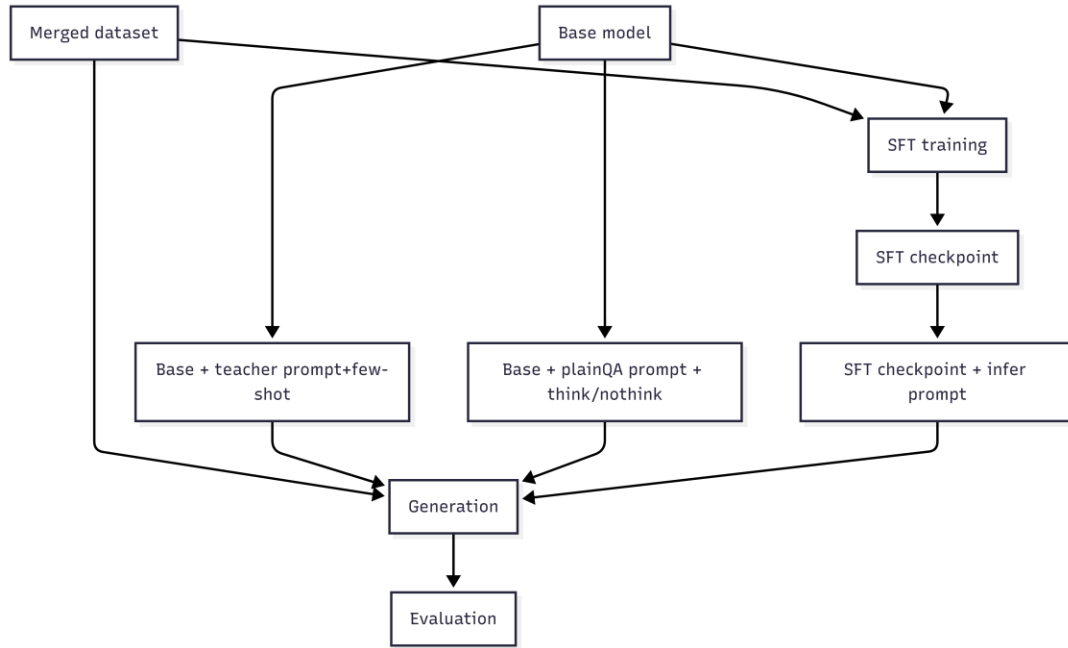


Figure 2 Workflow of training and evaluation



3.1.1 Natural-Language Formulation of the Task

The task studied in this thesis is a constrained knowledge + question QA task. The system receives a text input composed of externally provided knowledge together with a short-answer question. Its goal is to answer solely on the basis of the provided knowledge. When the provided knowledge is insufficient to support a verifiable short answer, the system should refuse to answer rather than fill in the missing information using parametric memory or world knowledge.

This task formulation carries two key implications.

First, the reliability criterion is anchored in the provided knowledge rather than in open-domain world truth. Therefore, even if the model output happens to be true in the external world, it is still considered insufficient under the present definition of grounded faithfulness if it is not directly supported by, or reasonably inferable from, the input knowledge.

Second, the thesis is not concerned with whether a retriever recalled the correct documents. Instead, under the assumption that the knowledge has already been provided, the focus is on whether the model can correctly judge answerability, identify key evidence, keep intermediate reasoning constrained, and perform a proper refusal when necessary.

For each sample, the system must first determine whether the input knowledge is sufficient to support the original question. If the answer is supportable, the system should locate supporting spans within the knowledge, organize the shortest necessary rationale, and output a short answer consistent with the answer style of the source dataset. If the knowledge is insufficient, the system should judge the question to be unanswerable and return a response using a predefined refusal template.

3.1.2 Answerable and Unanswerable Cases

Under the setting of this thesis, samples are divided into two basic cases.

- answerable: the given knowledge contains local spans sufficient to support the answer to the question;
- unanswerable: the given knowledge is insufficient to support a verifiable short answer to the original question.

Here, unanswerable does not mean that the question lacks an answer in reality; it means only that the knowledge currently provided is insufficient to support one.

3.2 Structured Output Protocol

3.2.1 Formal Protocol

This thesis adopts a structured output protocol with a fixed field order. Formally, the protocol can be represented as:

```
{
  "answerability": "answerable | unanswerable",
  "evidence": [{"quote": "..."}],
  "rationale": "...",
  "answer": "...",
  "confidence": "high | medium | low"
}
```

3.2.2 Semantics and Functional Justification of the Fields

The target output contains five fields: answerability, evidence, rationale, answer, and confidence.

1. **answerability:**

An enumerated field whose value must be either answerable or unanswerable, indicating whether the given knowledge is sufficient to answer the question.

This field explicitly separates the judgment of answerability from answer content. Because all subsequent tasks depend on this judgment, it is placed first in the protocol.

2. **evidence:**

A list of quote objects. When answerability is answerable, the model should extract one to four spans from the knowledge that contain the minimally sufficient facts needed to answer the question. When answerability is unanswerable, this field should be empty.

This field externalizes the specific knowledge spans selected by the model, allowing downstream checkers to inspect whether the evidence is copied from the original knowledge and whether it is sufficient to support the answer. During training, it also provides explicit supervision for how the student model should locate relevant content in long knowledge passages.

3. **rationale:**

A string. When answerability is answerable, the rationale should derive the answer using only the facts contained in evidence. When answerability is unanswerable, it should briefly state what information is missing, ambiguous, or contradictory in the knowledge, making the question unanswerable. The rationale must not be identical to the answer.

This thesis advocates minimal reasoning: the rationale should include only the smallest intermediate reasoning process necessary to reach the answer, so as to reduce the chance of hallucination. The field serves three purposes. First, it can improve answer interpretability for downstream use or end users. Second, compared with supervising only the final answer, the rationale provides richer linguistic supervision during training and helps the student model learn how to reason from evidence to answer. Third, it enables evaluation-time checking via an NLI-based verifier to determine whether the reasoning process from evidence to answer is actually supported, thereby detecting cases in which the final answer is accidentally correct but the intermediate reasoning has gone beyond the evidence.

4. **answer:**

A string. When answerability is answerable, it should be a phrase or single word sufficient to answer the question. When answerability is unanswerable, it should be one of the predefined refusal templates.

This field isolates a short answer sufficient to answer the question, which is consistent with the format of reference answers in the chosen dataset and facilitates automatic comparison. Separating the answer from the reasoning process also brings the benefits described above.

5. **confidence:**

An enumerated field whose value must be high, medium, or low, expressing the model's confidence in the reliability of the full output, whether it is an answer or a refusal.

Using discrete labels reduces learning and generation complexity. If the confidence field becomes reliable, it can later support system-level trade-offs between coverage and reliability.

In implementation, these field constraints are imposed in two ways: through prompt packaging at input time (see Appendix A, Prompt Templates) and through rule-based or model-based validation and filtering after generation (see Section 3.3, Validation, Checking, and Calibration Framework).

Overall, the purpose of the protocol is not merely to obtain parseable JSON. More importantly, it converts answer generation from an ambiguous mixed process into a sequence of constrained and interdependent subtasks. This motivation aligns with recent work on structured output, in which complex behavior is made explicit through schema-constrained generation: output structure determines not only format validity, but also the controlled behavior pattern that the model is required to learn.

3.3 Validation, Checking, and Calibration Framework

Qwen-family models occasionally introduce surface-form wrappers in raw outputs that are unrelated to semantics. These phenomena are not the focus of the present study, but they can interfere with subsequent parsing and comparison. Therefore, before later checks are applied, the system first canonicalizes the raw LLM output.

The raw text is first processed by reversing HTML entities, applying Unicode NFKC normalization, normalizing line breaks, removing zero-width characters, and trimming leading and trailing whitespace. The goal of this process is not to rewrite the semantic content of the model output, but to reduce interference from escape characters, full-width / half-width discrepancies, abnormal line breaks, and invisible characters without changing substantive meaning.

3.3.1 Overview of the Multi-Stage Checking Framework

The checkers in this thesis are not used only for final evaluation. The same judgment principles are reused at multiple points in the workflow, including data preparation, teacher-completion filtering, and unified evaluation of checkpoints and baselines after training, though different combinations are selected according to the needs of each stage. Overall, the thesis adopts a hybrid framework combining rule-based checkers and model-based checkers: the former ensure that input-output pairs satisfy predefined structure and budget constraints, while the latter handle semantic judgments that cannot be covered by simple rules.

1. Tokenizer-length check.

During data preparation, the system first uses the tokenizer of the student model to perform an explicit budget check on the prompt and completion lengths. The following code snippet checks whether the total token length of the knowledge and question, after being wrapped into the inference prompt, stays within the limit. This design not only saves resources, but more importantly prevents pseudo-samples caused by truncation of inputs or outputs from entering later training and evaluation stages. As a result, the conclusions become more reliable and more reproducible.

2. Parse check.

This check verifies whether the raw model output can be converted into a valid JSON object. The goals are:

- a) the output must be JSON;
- b) the JSON must contain all target fields;
- c) each field must have the correct data type;
- d) the values of answerability and confidence must lie within the allowed enumerations;

e) the content of rationale, answer, and quote must not be empty.

3. Protocol check.

This check verifies whether the structured object satisfies the protocol constraints.

The goals are:

- a) rationale must not be exactly the same as answer;
- b) when answerability == unanswerable, the evidence list must be empty and answer must use predefined refusal response;
- c) when answerability == answerable, the evidence list must contain one to four quote items.

4. Answerability-consistency check.

This check verifies whether the answerability field in the output matches the target label of the sample.

5. Evidence-substring check.

This check verifies whether each evidence quote, after normalization, is a strict substring of the knowledge. It does not judge whether the selected evidence is optimal or sufficient to support the answer; it is used only to detect free paraphrasing or fabricated quotations.

6. Correctness check

Agreement between answer and the reference answer is checked in two layers.

a) Correctness EM / F1 check.

The first layer is an exact-match (EM) / token-level F1 check against the reference answer. After answer normalization, the system checks whether the answer is an exact match or has token F1 above the threshold (default: 0.85).

The core logic is:

```
exact_match = _normalize_answer(answer) == _normalize_answer(reference_answer)
token_f1 = _token_f1(answer, reference_answer)
semantic_match = bool(exact_match or token_f1 >=
resolved_cfg.semantic_match_f1_threshold)
```

b) Correctness answer-equivalence-matcher check.

When the EM / F1 check in 6.1 fails, the sample is sent to the Answer Equivalence Matcher for secondary review.

This module inputs the question, the reference answer, and the candidate answer

into an answer-equivalence model, which outputs a matching score and decides semantic equivalence according to a threshold. This is necessary because short-answer QA often contains aliases, synonymous expressions, or semantically equivalent but surface-different forms; relying only on reference-answer EM / F1 would underestimate true correctness. The answer-equivalence model is designed precisely to alleviate this problem ([14], [15]).

It should be noted that, during SFT-example generation, the thesis uses only the first-layer EM / F1 filter in order to keep teacher supervision aligned as closely as possible to the source dataset's answer form. In final evaluation, however, the second-layer Answer Equivalence Matcher is enabled, because the concern at that stage is actual answer correctness rather than strict surface-form agreement with the reference answer.

7. Semantic check

a) Semantic NLI check.

Natural language inference (NLI) classifies a sentence pair into entailment, neutral, and contradiction. Its core task is to determine whether the hypothesis can be inferred from the premise ([13]). Correctness checking focuses only on the final short answer and therefore cannot constrain whether the reasoning process itself is supported by the evidence. For this reason, the thesis introduces an NLI-based verifier to examine whether the selected evidence entails the claims made by the output.

Two input-construction methods are used in this thesis:

Protocol mode (the main evaluation mode in this thesis)

premise: constructed from the selected knowledge obtained by concatenating evidence, together with the question;

hypothesis: a declarative sentence composed from rationale and answer.

Flat-output mode (used for non-protocol baselines)

premise: constructed from the full knowledge together with the question;

hypothesis: the raw output text itself.

Margin and temperature scaling:

The raw NLI output consists of probabilities for entailment, neutral, and

contradiction. In this thesis, both neutral and contradiction are treated as signals of unsupportedness or conflict, so the three-way output must be converted into a binary decision. The decision margin is defined as:

$$m = \frac{\log p(e)}{T} - \max\left(\frac{\log p(n)}{T}, \frac{\log p(c)}{T}\right)$$

When $margin \geq margin_threshold$, the output is judged as supported. The margin requires entailment to have a sufficiently strong advantage over neutral or contradiction, making non-hallucination decisions more robust. Temperature scaling does not change ranking, but rescales distribution sharpness so that margins become more comparable across samples.

b) Semantic QA-consistency check.

This is a supplement to NLI-based judgment. It identifies several question types from question prefixes and then uses spaCy named-entity recognition and numeric extraction to perform rule-based answer-type validation. The covered types are:

- i. who questions require the answer to contain an entity of type PERSON or ORG;
- ii. where questions require the answer to contain an entity of type GPE or LOC;
- iii. when questions require the answer to contain an entity of type DATE or TIME, or at least a year;
- iv. how many / how much questions require the answer to contain a numeric value.

This check captures failure cases in which the answer type is inconsistent with the question type beyond what NLI alone can detect. A failure on this check also causes the overall semantic check to fail.

8. DeepEval Hallucination Metric.

In addition to local rule-based and local model-based checkers, the thesis introduces the DeepEval Hallucination Metric as an LLM-as-a-judge signal during the final test stage.

HallucinationMetric uses an LLM to determine, for each context in contexts,

whether the actual_output contains a contradiction with that context. The score is computed as ([16]):

$$hallucination = \frac{\text{number of contradicted context}}{\text{total number of context}}$$

In this thesis, DeepEval is applied to the same objects as the NLI checker:

For protocol outputs, rationale + "Therefore the answer is " + answer is used as actual_output, and the selected evidence is used as context.

For non-protocol outputs, the raw output text is used as actual_output, and the full knowledge is used as context.

In both cases, question is used as input.

Compared with NLI, this method is closer to how a human reviewer assesses hallucination from the perspective of the full text. Because it requires calls to a strong model, however, it is more expensive, and is therefore used only as a supplementary signal during final test-stage review rather than as part of the training-data filtering pipeline.

3.3.2 Calibration Procedures

The temperature and decision-threshold parameters of the semantic NLI judge, as well as the decision threshold of the Answer Equivalence Matcher, are calibrated. The calibration goal is to find parameter settings that maximize agreement with manually annotated gold labels, measured by F1.

The calibration data are constructed according to the following principles:

1. run the checkers once under the default configuration and collect the samples they actually judge;
2. perform stratified sampling on that set and obtain blind human gold labels;
3. search over parameters while keeping the gold labels fixed, and report accuracy, precision, recall, F1, Cohen's kappa, and confusion-matrix statistics (tp/tn/fp/fn) to assess agreement between the checker and human annotation.

Search strategy:

Semantic NLI:

- Temperature: use the manually annotated set as a validation set and minimize negative log-likelihood (NLL).

- Margin threshold: after fixing the temperature, enumerate the margin threshold within a predefined range using a predefined step size, compute agreement with the human labels, and choose the threshold that yields the best F1.

Answer Equivalence Matcher:

- Enumerate matcher_threshold on the manually annotated set and choose the threshold that yields the best F1.

3.4 Data Construction Strategy

3.4.1 Construction of Answerable Instances

The raw data source used in this thesis is the HotpotQA dataset, whose per-instance structure is as follows:

```
{
  "id": "...",
  "question": "...",
  "answer": "...",
  "type": "...",
  "level": "...",
  "supporting_facts": {
    "title": [...],
    "sent_id": [...]
  },
  "context": {
    "title": [...],
    "sentences": [[...], [...], ...]
  }
}
```

Since question and answer are already available, only knowledge construction is discussed here.

context is the full context for the sample and consists of multiple Wikipedia entries, each containing a title and an ordered list of sentences. supporting_facts provides the gold supporting evidence required to answer the question. These annotations are sentence-level rather than paragraph-level and locate each supporting fact by a (title, sent_id) pair within the context. To preserve answer-supporting information while controlling the amount of irrelevant context, this thesis expands around the original supporting-fact sentences by sampling a window of neighboring sentences, thereby producing a context that remains answerable while including some distractor information as knowledge ([12]).

3.4.2 Deriving Unanswerable Instances from Answerable Ones

Because HotpotQA does not itself provide annotated unanswerable instances, the unanswerable samples in this thesis are derived from the answerable scaffolds constructed in the previous step.

Specifically, the system randomly selects one or more original supporting sentences from an answerable sample and replaces them with nearby non-supporting sentences, preferring candidates from the same document that are close to the original supporting sentence. If no suitable candidate exists nearby in the same document, adjacent sentences from the beginning or end of neighboring documents are considered. The goal is to generate negative samples that remain highly similar to the answerable scaffold in local style while breaking the support chain required for the original answer.

Because the human supporting-fact annotations in HotpotQA are not guaranteed to exhaust all potentially supporting information, the modified knowledge may still accidentally retain enough support. Therefore, the data-construction stage includes an NLI prefilter. For samples intended to be unanswerable, the post-replacement knowledge + question is used as the premise and the original answer as the hypothesis, and the sample is kept only if the NLI verifier judges it unsupported. For yes/no samples, the flipped answer is additionally required to be unsupported as well. This helps ensure that the constructed sample is genuinely unanswerable.

3.4.3 Prompt-Length Filtering Before Teacher Generation

Before teacher generation, all constructed samples are wrapped using the actual inference prompt format and filtered according to input token length. Samples exceeding the prompt budget are removed. This prevents knowledge loss caused by truncation in later stages and keeps teacher generation, student training, and evaluation aligned to the same input interface.

3.4.4 Answerability Construction Ratio

The construction-label ratio for answerability-oriented data generation is set to answerable : unanswerable : both = 0.7 : 0.1 : 0.2, with three purposes:

First, answerable samples remain the dominant component so that the main distribution of HotpotQA as a QA dataset is preserved.

Second, a certain proportion of derived unanswerable samples is retained to train refusal behavior and answerability judgment.

Third, the both category generates paired positive and negative samples from the same source scaffold, increasing the model's sensitivity to boundary differences.

3.4.5 Independent Stable Split Assignment

There are in fact two independent stable partitioning mechanisms in the data-construction pipeline.

The first is the split into `train_sft_raw` / `validation` / `test`.

The second is the assignment of the construction label `answerable` / `unanswerable` / `both`.

Both are assigned using stable hash functions based on `source_id` together with a salt.

`train_sft_raw` samples come from the official HotpotQA train split, while `validation` and `test` are obtained by repartitioning the official validation split. This design ensures reproducible split assignment and avoids cross-stage data leakage.

3.5 Teacher Supervision Generation and Selection

3.5.1 Teacher Prompt and Infer Prompt

This thesis uses two different prompt interfaces for teacher-supervision generation and student inference.

The teacher prompt provides a complete protocol description, field order, hard constraints, predefined refusal response, and the JSON schema. For answerable samples, it additionally includes the Reference answer and requires the final answer to remain semantically consistent with that reference answer. By contrast, the infer prompt retains only the minimal task description and rule list and does not expose the reference answer. This design serves two distinct goals.

The teacher prompt is intended to generate structured supervision of the highest possible quality, and therefore allows stronger external constraints and a gold anchor.

The infer prompt, in contrast, is intended to test whether the student model has truly learned protocol behavior and to reduce deployment cost in realistic use.

3.5.2 Teacher Generation Strategy

For answerable samples, the system generates three candidates per sample.

For unanswerable samples, it generates only one candidate.

The reason is that answerable candidates must subsequently pass multiple filters, including EM / F1, evidence-substring checking, and semantic NLI, so the pass rate of a single candidate is limited and multiple candidates leave room for filtering and ranking. Unanswerable samples, by contrast, are structurally simpler and usually pass later rule-based checks with a higher rate. Moreover, if the teacher answers an actually unanswerable sample as answerable, that failure itself can serve as an additional filtering signal to ensure unanswerability. It is therefore unnecessary to generate multiple candidates for unanswerable instances.

3.5.3 Basic Validity Checks

All samples generated by the teacher method above must pass the following basic checks:

- Pass tokenizer-length check;
- Pass parse check;
- Pass protocol check;
- Pass answerability-consistency check.

3.5.4 Validation and Selection for Answerable Candidates

Answerable candidates must further satisfy the following conditions:

- Pass the evidence-substring check;
- Pass the correctness EM / F1 check;
- Pass the semantic check;

Among candidates that satisfy these hard conditions, selection is then performed using several soft signals, including:

- higher semantic margin (higher semantic reliability);
- a higher proportion of evidence quotes that fall within supporting facts (greater alignment between evidence selection and original dataset annotations);
- higher correctness token F1 (closer agreement between the answer and the reference answer);
- fewer evidence quotes (preference for candidates that reach the correct answer with less evidence).

3.5.5 Validation and Selection for Unanswerable Candidates

Teacher completions for unanswerable samples do not require additional conditions beyond the basic validity checks, because the legality of their fields is already fully verified at the basic-validation stage.

Importantly, NLI-based semantic checking is not suitable for unanswerable samples, because models of that kind are neither designed nor trained to judge the reliability of refusal behavior.

3.5.6 Post-hoc Confidence Relabeling

The raw confidence labels produced by the teacher are overwhelmingly concentrated in high, according to sampling observations. This may be because the teacher model is too strong to produce useful variation in difficulty, or because instruction tuning and preference alignment make the teacher overconfident. In either case, the original distribution does not provide a useful training signal.

Therefore, this thesis applies post-hoc confidence relabeling to obtain a more balanced distribution. For answerable samples, high / medium / low are assigned by evenly dividing the validation-report semantic margin into quantile buckets. For unanswerable samples, confidence is assigned by inverse quantile bucketing based on the NLI prefilter score from the construction stage.

3.6 Student Model Training

3.6.1 Training Data Construction

Training data are built from the structured samples retained after teacher-supervision generation and validation / selection. The structured object is first restored and revalidated, and then serialized into canonical JSON. In the training target, the original confidence field is replaced with the `derived_confidence` produced in the validation report, and this becomes the completion. The same knowledge-question pair is then rewrapped with the infer prompt to form the training prompt. Consequently, every training sample is represented uniformly as a prompt-completion pair, where the input side uses the minimal inference interface and the output side uses the standard structured answer under protocol constraints.

This allows the student model to learn how to generate protocol outputs under the real inference interface.

3.6.2 SFT Trainer

This thesis uses the standard supervised fine-tuning workflow represented by Hugging Face TRL's SFTTrainer: it takes prompt-completion data as input, directly supports this data format, and by default computes loss only on the completion portion ([17]).

This tool packages standard procedures for instruction / prompt-completion fine-tuning, including preprocessing, tokenization, loss masking, and the training loop, into a unified interface. As a result, the thesis can focus on training objectives and data design rather than reimplementing a generic training scaffold.

3.6.3 LoRA with PEFT

During training, this thesis uses the PEFT framework to perform parameter-efficient fine-tuning with LoRA.

The basic idea of LoRA is to freeze the original weights of the pretrained model and inject low-rank trainable matrices into target layers so that task adaptation can be approximated with a relatively small number of new parameters ([18]). This greatly reduces the number of trainable parameters and memory usage and usually does not introduce extra inference latency, making it well suited to task-specific adaptation of a base model under moderate computational budgets.

Using PEFT's LoraConfig, this thesis injects LoRA adapters into q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, and down_proj, training only the newly added parameters while keeping the main body of the base model fixed. After training, what is saved is the LoRA adapter checkpoint together with the tokenizer. During validation and test, different LoRA checkpoints are mounted on the same base model for generation and evaluation, thereby forming a comparable training trajectory.

3.7 Evaluation Method and Baseline Setup

3.7.1 Validation-Stage Checkpoint Evaluation

After SFT training, all checkpoints saved during training are evaluated on the validation set in two ways.

The first is SFT loss evaluation. The system recomputes token-level negative log-likelihood on the validation prompt-completion pairs, using the same tokenization, completion-only mask, and length-handling rules as in training. This produces a loss curve over training steps and indicates whether the model is still learning or whether returns have begun to saturate.

The second is generation-based validation evaluation. For each checkpoint, the system generates structured outputs on the validation set under the formal comparison setting and then applies parse checking, protocol checking, answerability-consistency checking, evidence-substring checking, correctness checking, and semantic checking in sequence. This supplements pure loss metrics: a lower SFT loss does not automatically guarantee legal protocol fields, answerability reliability, or stronger evidence support. Therefore, validation must consider both fit to the supervision target and real generated behavior. This thesis selects the best checkpoint on the basis of these validation curves and checker outputs; the exact metrics and selection rule are described in Chapter 4.

3.7.2 Test-Stage Comparative Evaluation and Baselines

After the best checkpoint is selected on the validation set, the system evaluates it on the test set together with three types of baselines so as to assess the true benefits brought by the protocol and SFT training. To ensure reproducibility and comparability, all model pathways entering the formal comparison use deterministic decoding, namely `temperature=0` and `top_p=1`. Teacher generation is used only for supervision-data construction and therefore is not part of the comparison setting.

The adopted baselines are of three types:

First, base flat no-think: the base model directly produces an ordinary natural-language answer under the Plain Knowledge QA Prompt with explicit refusal encouragement, without enabling thinking mode. This corresponds to the default flat QA setting of the base model on the task studied here and serves as the primary baseline, because the protocol design and the subsequent SFT training are both, in effect, additional constraints imposed on top of this default setting. The other two baselines are auxiliary comparisons that help explain the sources of gain.

Second, base protocol: the base model directly produces protocol outputs under a strong teacher prompt + few-shot examples setting. This baseline is used to compare pure prompt engineering with a protocol that has been internalized through SFT.

Third, base flat think: the prompt is the same as that of the primary baseline, but the base model's think mode is enabled. This baseline is used to compare the proposed protocolized grounded generation with the base model's generic think mode, thereby probing the difference between generic reasoning and the task-specific protocol defined in this thesis.

3.7.3 Unified Evaluation Path for Protocol and Flat Outputs

Under the setting above, the test stage yields two types of outputs: protocol output, namely explicit JSON outputs that follow the structured protocol, and flat output, namely ordinary free-form text answers. To derive comparable evaluation signals from these two modes, the thesis uses an evaluation path that is partly shared and partly differentiated. For protocol output, the system can directly apply parse checking, protocol checking, and evidence-substring checking to the raw output and can further evaluate answerability, correctness, semantic support, and DeepEval hallucination. Because the output already contains explicit answerability, evidence, rationale, and answer fields, most judgments can be made directly on the protocol object.

For flat output, the system uses an external LLM to extract a comparable short answer and detect refusal from free-form text, so that an answerability label can be recovered and correctness can be compared against the gold answer. At the same time, the original free-form text itself is still used as the object of semantic and DeepEval hallucination evaluation, against the full knowledge, to determine whether it contains content not supported by context. This yields two comparable groups of signals: one based on the extracted short answer and aligned to the gold answer, and the other based on the original model output and used to assess whether the overall text is supported by the knowledge. In summary, the two output types share the checks for answerability, correctness, semantic support, and DeepEval hallucination. Only protocol output additionally receives parse validity, protocol validity, and evidence-substring checks. The validation stage uses all of the above except DeepEval for checkpoint selection. Because DeepEval is more

costly, it is used only as a supplementary LLM-as-a-judge review signal during the final test stage.

4 Experimental Setup

4.1 Dataset and Split

4.1.1 Dataset Choice

The main experimental data source in this thesis is the distractor subset of HotpotQA ([12]). This dataset provides multi-hop short-answer QA, sentence-level supporting-fact annotations, and contexts composed of multiple document entries, making it highly suitable for the knowledge grounded multi-hop QA task studied here. Compared with datasets that provide only long documents without explanatory supervision, HotpotQA's supporting-fact annotations are useful both for constructing compact knowledge inputs and for deriving answerable / derived-unanswerable boundary samples.

In the implementation, the code by default reads 20,000 samples from the official train split and 2,000 samples from the official validation split. This setting is intended to control experimental scale and does not change the basic definition of the data source.

4.1.2 Construction Parameters

The key source-stage data-construction parameters are shown in Table 1.

Table 1 Source-stage data construction parameters

Parameter Category	Parameter	Value
answerable construction	window_size	1
answerable construction	max_supporting_facts	4
answerable construction	include_title_prefix	True
unanswerable construction	replace_supporting_facts_min	1
unanswerable construction	replace_supporting_facts_max	1
unanswerable construction	same_doc_candidate_radius	1
unanswerable construction	allow_same_doc_non_adjacent	True
unanswerable construction	adjacent_doc_sentence_limit	1
prompt budget	max_prompt_tokens	512
completion budget	max_completion_tokens	512
construction-label ratio	answerable : unanswerable : both	0.7 : 0.1 : 0.2

source prefilter	enable_unanswerable_nli	True
------------------	-------------------------	------

window_size=1 and max_supporting_facts=4 keep the knowledge compact.

replace_supporting_facts_min=max=1 constrains derived unanswerable construction to minimal perturbation, so that the experiment focuses more sharply on whether the model is sensitive to a broken support chain.

The prompt and completion budgets are applied so as to avoid introducing hidden truncation effects across stages as additional confounders. The construction-label ratio of 0.7 : 0.1 : 0.2 keeps answerable samples dominant while retaining enough derived unanswerable instances and paired boundary samples.

4.1.3 Train / Validation / Test Split

Data partitioning is determined jointly by two stable mechanisms. First, all samples from the official train split are mapped to train_sft_raw, while the official validation split is repartitioned into downstream validation and test at a stable hash ratio of 0.5 : 0.5.

Second, each source row is independently assigned a construction label from answerable / unanswerable / both. Because the two partitioning schemes are independent, the answerability composition of the training data can be controlled without introducing data leakage.

After teacher supervision is constructed, the train and validation splits are organized into SFT records with completions, while the test split retains only knowledge and questions. The sampling caps are shown in Table 2.

Table 2 Split-specific sampling caps

Split	Total Cap	answerable Cap	unanswerable Cap
train	10,000	8,000	2,000
validation	500	400	100
test	500	400	100

4.2 Prompt Templates

4.2.1 Prompt Families Used in the Experiments

Four prompt families are used in the experiments. They share the same knowledge + question input, but differ in how they provide structural constraints, auxiliary supervision, and demonstration support. Full templates are given in Appendix A.

- **teacher prompt**

- **Included information:**

- Complete protocol specification
 - Field order
 - Evidence-copying constraint
 - Minimal-reasoning constraint
 - predefined refusal response
 - JSON schema
 - knowledge
 - question
 - For answerable samples, also includes reference answer

- **Stage of use:** teacher-supervision generation

- **Function:** Provides the strongest external constraints to the teacher model in order to obtain high-quality structured supervision

- **teacher prompt + few-shot**

- **Included information:**

- All information in the teacher prompt
 - Two complete solved examples

- **Stage of use:** base protocol baseline

- **Function:** Maximizes the stability with which the base model follows the protocol format and refusal norm without training

- **infer prompt**

- **Included information:**

- Field order
 - Minimal necessary rules
 - Canonical refusal
 - knowledge
 - question

- **Stage of use:**
 - SFT training
 - Checkpoint inference
 - Final structured evaluation
- **Function:** Serves as the unified interface for the student model and tests whether the protocol has truly been internalized rather than merely triggered by a long prompt
- **plain knowledge QA prompt**
 - **Included information:**
 - Task instruction to answer using only the provided knowledge
 - Explicit refusal rule
 - knowledge
 - question
 - **Stage of use:**
 - base flat no-think
 - base flat think baselines
 - **Function:** Builds flat-task baselines without the structured protocol so as to compare the gains brought by protocolized supervision

In terms of information strength, teacher prompt and teacher prompt + few-shot are both significantly stronger than infer prompt. The former exposes the full schema, while the latter further adds demonstrations. By contrast, plain knowledge QA prompt does not constrain the evidence, rationale, or confidence fields and serves as a control baseline with the same knowledge input but a different output protocol.

4.3 Models and Inference Configurations

4.3.1 Qwen3-4B

The Qwen3 family is a set of open-weight models released by the Qwen team at Alibaba Cloud. It provides strong foundations in reasoning, instruction following, and multilingual understanding, and is distributed under the Apache License 2.0 ([19]). Its open availability, coverage of multiple model sizes, and strong benchmark performance make it an appropriate and reproducible research backbone.

The family also supports both think and no-think modes. In this study, those two modes are used to form two baseline groups for comparison so that the gains from the proposed protocol can be compared against both direct answering and generic reasoning.

This thesis adopts Qwen3-4B as the training target and reference baseline mainly for the following reasons:

1. it has reasonably strong basic language ability, instruction-following ability, and some reasoning capability, yet its capacity is not so saturated that learning gains from structured supervision become unobservable;
2. it has sufficient parameter capacity to support structured protocol learning and the basic expressive requirements of multi-hop QA;
3. with PEFT methods, repeated training and inference experiments can be conducted at manageable cost.

4.3.2 Teacher and Auxiliary Models

Teacher-supervision generation is performed through the official Alibaba Cloud API using qwen3.5-plus, one of Qwen's flagship models released in 2026. It offers strong instruction-following and reasoning ability and can stably generate outputs that satisfy the protocol while providing high-quality evidence selection and grounded reasoning as training signals.

In addition, short-answer extraction and refusal detection for flat baselines are handled by qwen-plus. Because this auxiliary model only serves a narrow post-processing role, stable instruction following and lower calling cost are prioritized.

DeepEval uses gpt-5.2 as the judge model. This strong judge model can provide a reliable holistic hallucination-review signal, which is consistent with DeepEval's recommended usage. Using model families different from those used for training-data generation and primary evaluation also helps reduce evaluation bias.

4.3.3 Inference Configuration Table

The key inference parameters for each pathway are shown in Table 3.

Table 3 Inference configurations

Pathway	model	prompt Template	few-shot	max_new_ tokens	temperatu re	top_p	enable_t hink	chat_template
teacher	qwen3.5-plus	teacher prompt	0	512	0.2	0.95	True	N/A

sft checkpoint	Qwen/Qwen3-4B + LoRA ckpt	infer prompt	0	512	0.0	1.0	False	True
base protocol	Qwen/Qwen3-4B	teacher prompt + few-shot	2	512	0.0	1.0	False	True
base flat no-think	Qwen/Qwen3-4B	plain knowledge QA prompt	0	512	0.0	1.0	False	True
base flat think	Qwen/Qwen3-4B	plain knowledge QA prompt	0	1024	0.0	1.0	True	True

All pathways entering the formal comparison use deterministic decoding, namely temperature=0 and top_p=1, in order to improve reliability and reproducibility of prompt-generated results. The chat_template is enabled uniformly to keep generation behavior stable. Think-mode generation uses a larger output budget so that sufficient space is available for reasoning.

4.4 Training Configuration

4.4.1 SFT Hyperparameters

The SFT hyperparameters are shown in Table 4.

Table 4 SFT hyperparameters

Parameter	Value
train_epochs	2.0
learning_rate	2e-4
train_batch_size	4
eval_batch_size	8
gradient_accumulation_steps	8
max_length	1024
save_steps	50
save_total_limit	20
logging_steps	10

Most of these parameters follow stable default choices; the SFT hyperparameters themselves are not treated as the research object in this thesis. A learning rate of 2e-4,

combined with LoRA on a 4B-scale model, is a relatively conservative and commonly used setting.

4.4.2 LoRA Configuration

The LoRA-related parameters are shown in Table 5.

Table 5 LoRA configuration

Parameter	Value
lora_r	16
lora_alpha	32
lora_dropout	0.05
lora_target_modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj

These settings likewise follow stable default values. $r=16$ and $\alpha=32$ provide moderate adapter capacity, allowing the model to learn protocol consistency and content consistency, while $\text{dropout}=0.05$ supplies lightweight regularization. The target modules cover the major projection layers in both attention and the MLP. The goal is to absorb structured behavior patterns at low training cost rather than to explore new PEFT designs. Training and inference both use the Qwen/Qwen3-4B tokenizer. Because prompt and completion are already strictly filtered for length during data construction, additional truncation almost never occurs during training, and truncation strategy is therefore not treated as an independent experimental variable.

4.5 Checker Parameters and Calibration

This section records the model choices and parameter settings that affect the experimental results, following the order in which the checkers are introduced in Section 3.3. Checkers without tunable parameters are described only briefly.

4.5.1 Tokenizer-Length Check

Tokenizer-length checking uniformly uses the tokenizer of the student model, Qwen/Qwen3-4B, so that pre-training filtering, training-time packing, and evaluation-time budget checks all operate on the same underlying model interface and therefore share the same length constraints.

4.5.2 Parse, Protocol, Answerability, and Evidence Substring Checks

The parse check, protocol check, answerability-consistency check, and evidence-substring check are all deterministic rule-based checks. They use no additional models and contain no threshold parameters requiring calibration.

4.5.3 Correctness Checks

The first layer of correctness checking is an EM / token-level F1 comparison against the reference answer. This thesis sets the token-F1 threshold to 0.85. In short-answer QA, this is already a high threshold, meaning that the candidate answer must agree with the reference answer on nearly all content words, usually allowing only differences in articles, singular-plural variation, mild inflectional changes, or a small number of function words. Therefore, when token F1 ≥ 0.85 , the compared strings can be regarded as essentially identical in surface form. Cases below the threshold are then handed to the second semantic-equivalence review layer.

When strict EM / F1 checking fails, the system invokes an answer-equivalence matcher for secondary review. The checker uses zli12321/answer_equivalence_roberta-large, and its threshold is selected by grid search on a manually annotated set. The corresponding settings and results are shown in Table 6.

Table 6 Answer equivalence matcher calibration results

Checker	Model Used	Search Range	Final Threshold	Agreement with Human Annotation
Answer equivalence matcher	zli12321/answer_equivalence_roberta-large	0.1 to 0.95, step size 0.05	0.27	accuracy=0.87, F1=0.8860 (200 manually annotated samples)

This setup indicates that the matcher should serve as a supplementary reviewer after strict matching fails rather than as a replacement for strict matching itself. Its optimal threshold is relatively low, suggesting that aliases, abbreviations, and small wording differences are common in short-answer QA. Insisting on a much higher threshold would underestimate true correctness. Its relatively high F1 and accuracy indicate that the model has good practical discriminative power on the manually annotated set.

4.5.4 Semantic Checks

The main semantic checker is a local NLI verifier. This thesis uses MoritzLaurer/DeBERTa-v3-large-mnli-fever-anli-ling-wanli as the three-way NLI model and calibrates its outputs using temperature scaling and a margin decision rule. The model can be run locally in batches and is suitable as an evidence-supported entailment verifier ([21]). The search space and final parameters are shown in Table 7.

Table 7 Semantic NLI calibration results

Checker	Model Used	Search Parameter	Search Range	Final Value	Agreement with Human Annotation
Semantic NLI	MoritzLaurer/DeBERTa-v3-large-mnli-fever-anli-ling-wanli	temperature	0.05 to 5.0, step size 0.01	2.17	accuracy=0.8575, F1=0.9182
		margin_threshold	0.0 to 1.0, step size 0.05	0.4	

The optimal temperature=2.17 indicates that the original three-way probability distribution is relatively overconfident and should be softened before margin comparison. margin_threshold=0.4 means that entailment must maintain a sufficiently clear advantage over neutral and contradiction before the sample is judged supported. The resulting accuracy and F1 are both high, indicating that the checker provides an effective semantic-support signal.

4.5.5 DeepEval Hallucination Check

The DeepEval hallucination check uses gpt-5.2 as the judge model and adopts threshold=0.5, which is consistent with the recommended DeepEval setting.

4.6 Metric Set and Checkpoint Selection

To unify notation, let the total number of evaluation samples be N . For the i -th sample, define:

$$P_i = 1[\text{parse_ok}]$$

$$U_i = 1[\text{usable}]$$

$$A_i = 1[\text{usable} \wedge \text{answerability_match}]$$

$$G_i^{ans} = 1[\text{gold answerable}]$$

$$G_i^{una} = 1[\text{gold unanswerable}]$$

$$C_i = 1[\text{correctness_reviewed_success}]$$

$$S_i = 1[\text{semantic_decision} = \text{yes}]$$

$$D_i = 1[\text{deepeval_pass}]$$

Here, e2e means whether, starting from the raw model output and passing through all gates permitted by the current evaluation path, the sample reaches a final state that is acceptable to an end user.

4.6.1 Primary End-to-End Metrics

e2e_answerability_accuracy:

$$\frac{1}{N} \sum_{i=1}^N A_i$$

Meaning: the proportion of samples for which the model, from raw output to final usable state, at least makes the answerability judgment correctly.

e2e_reviewed_correctness_success_rate:

$$\frac{1}{N} \sum_{i=1}^N (G_i^{una} A_i + G_i^{ans} A_i C_i)$$

Meaning: the primary user-facing success metric for whether the system ultimately gives an acceptable response. For gold unanswerable samples, only a correct refusal is required; for gold answerable samples, the answer must be correct after answerability has been judged correctly.

e2e_semantic_success_rate:

$$\frac{1}{N} \sum_{i=1}^N (G_i^{una} A_i + G_i^{ans} A_i S_i)$$

Meaning: the end-to-end success rate with respect to evidential support. Together with reviewed correctness, it distinguishes between "the answer is correct" and "the answer is supported by evidence."

e2e_deepeval_pass_rate:

$$\frac{1}{N} \sum_{i=1}^N (G_i^{una} A_i + G_i^{ans} A_i D_i)$$

Meaning: the end-to-end pass rate when an LLM-as-a-judge hallucination review is applied. This metric is reported only on evaluation paths that run DeepEval.

e2e_confidence_resolution_reviewed_correctness:

Let Y_i^{rc} denote the binary result of e2e_reviewed_correctness_success, and let the confidence bucket be $k \in \{\text{low}, \text{medium}, \text{high}\}$. Then:

$$\frac{\sum_k \frac{n_k}{N} (\overline{Y}_k^{rc} - \overline{Y}^{rc})^2}{\overline{Y}^{rc}(1 - \overline{Y}^{rc})}$$

Meaning: the extent to which self-reported confidence distinguishes success from failure in final reviewed correctness, rather than merely ranking already-passed samples.

e2e_confidence_resolution_semantic:

Let Y_i^{sem} denote the binary result of e2e_semantic_success. Then:

$$\frac{\sum_k \frac{n_k}{N} (\overline{Y}_k^{sem} - \overline{Y}^{sem})^2}{\overline{Y}^{sem}(1 - \overline{Y}^{sem})}$$

Meaning: the extent to which confidence distinguishes final semantic success. Both resolution metrics adopt the normalized resolution term from Murphy decomposition ([20]).

4.6.2 Secondary Diagnostic Metrics

usable_rate:

$$\frac{1}{N} \sum_{i=1}^N U_i$$

Meaning: the proportion of outputs that become valid samples that can continue through evaluation. This is a prerequisite for all downstream success metrics.

parse_ok_rate:

$$\frac{1}{N} \sum_{i=1}^N P_i$$

Meaning: a diagnostic of whether the model output can first be parsed as JSON.

protocol_ok_rate_given_parse_ok:

Let $R_i = \mathbf{1}[\text{protocol_ok}]$. Then:

$$\frac{\sum_{i=1}^N P_i R_i}{\sum_{i=1}^N P_i}$$

Meaning: conditional on successful parsing, whether the field-level and structural constraints of the protocol are satisfied.

evidence_substring_rate_on_usable_pred_answerable:

Let

$$M_i = 1[\text{usable} \wedge \text{predicted answerable}]$$

$$E_i = 1[\text{all evidence quotes are substrings of knowledge}]$$

Then:

$$\frac{\sum_{i=1}^N M_i E_i}{\sum_{i=1}^N M_i}$$

Meaning: when the model actually predicts an answerable case, whether the copied evidence is legally grounded in the knowledge.

answerable_reviewed_correctness_success_rate:

$$\frac{\sum_{i=1}^N G_i^{ans} A_i C_i}{\sum_{i=1}^N G_i^{ans}}$$

Meaning: the final correctness performance on the answerable subset in isolation.

answerable_semantic_success_rate:

$$\frac{\sum_{i=1}^N G_i^{ans} A_i S_i}{\sum_{i=1}^N G_i^{ans}}$$

Meaning: the final semantic-support performance on the answerable subset in isolation.

unanswerable_success_rate:

$$\frac{\sum_{i=1}^N G_i^{una} A_i}{\sum_{i=1}^N G_i^{una}}$$

Meaning: the model's ability to refuse and make correct answerability judgments under knowledge-insufficient conditions.

4.6.3 Checkpoint Selection Rule

Checkpoint selection follows a hard-constraints-first, ranking-second rule.

The hard constraints are:

parse_ok_rate ≥ 0.95

protocol_ok_rate_given_parse_ok ≥ 0.98

evidence_substring_rate_on_usable_pred_answerable ≥ 0.95

If one or more checkpoints satisfy all hard constraints, they are ranked in the following order:

1. highest e2e_reviewed_correctness_success_rate;
2. highest e2e_answerability_accuracy;
3. highest e2e_semantic_success_rate;
4. lowest validation mean_loss;
5. smallest model_step.

If no checkpoint satisfies the hard constraints, the same ranking rule is applied to all checkpoints, and the result is treated as a fallback selection.

The rationale is to ensure first that the structure is parseable, the protocol is valid, and evidence references are trustworthy, and then to prioritize the checkpoint that is most beneficial to final user-facing correctness. Answerability and semantic success are used to further distinguish close candidates, while validation loss serves only as a final tie-breaker.

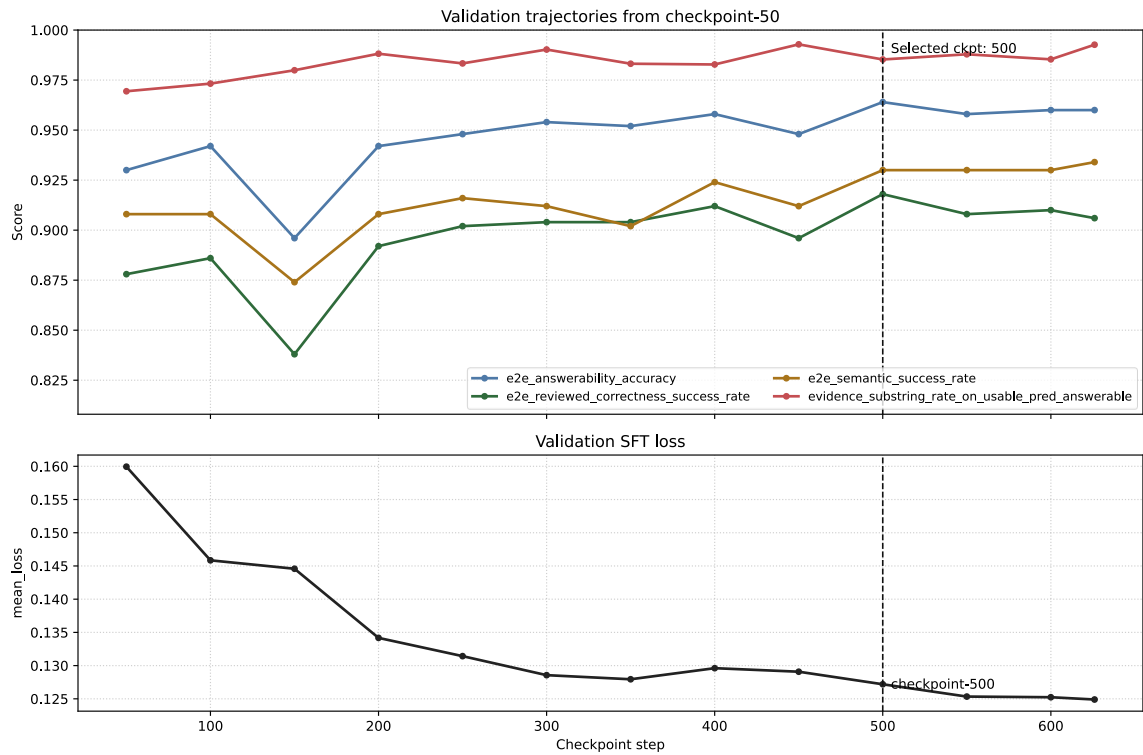
5 Results and Analysis

5.1 Validation Results and Checkpoint Selection

Table 8 Key validation metrics and checkpoint selection results

model _step	mean_lo ss	e2e_reviewed_ correctness_su ccess_rate	e2e_answerabil ity_accuracy	e2e_semantic _success_rate	evidence_substring_rate_on _usable_pred_answerable	unanswerable _success_rate
50	0.1599	0.878	0.930	0.908	0.969	0.700
100	0.1458	0.886	0.942	0.908	0.973	0.800
150	0.1446	0.838	0.896	0.874	0.980	0.500
200	0.1342	0.892	0.942	0.908	0.988	0.740
250	0.1314	0.902	0.948	0.916	0.983	0.770
300	0.1286	0.904	0.954	0.912	0.990	0.830
350	0.1279	0.904	0.952	0.902	0.983	0.800
400	0.1296	0.912	0.958	0.924	0.983	0.860
450	0.1291	0.896	0.948	0.912	0.993	0.770
500	0.1272	0.918	0.964	0.930	0.985	0.870
550	0.1253	0.908	0.958	0.930	0.988	0.830
600	0.1252	0.910	0.960	0.930	0.985	0.850
626	0.1249	0.906	0.960	0.934	0.993	0.850

Figure 3 Validation curves



5.1.1 Validation-Stage Trend Analysis and Checkpoint Selection

On the validation set, this thesis examines two kinds of signals simultaneously: token-level SFT loss recomputed over prompt-completion pairs, and protocol-aware validation metrics derived from actual generated outputs. The former reflects fit to the supervision target, whereas the latter reflects whether the model satisfies the actual requirements under the formal inference interface.

The loss curve shows that the model obtains substantial gains early in training. Compared with the base state, validation mean_loss drops rapidly over the first several checkpoints, indicating that the student quickly learns the basic distribution of structured completions. The loss continues to decline afterward, but with a much smaller marginal rate of improvement, exhibiting a typical pattern of fast early convergence followed by diminishing returns.

The generation-based validation trends show that structure-legality metrics stabilize early in training. After step 100, parse_ok_rate, protocol_ok_rate_given_parse_ok, and usable_rate are already close to saturation, indicating that the student learns very early how to generate protocol outputs that are parseable and field-consistent. At the same time, evidence_substring_rate_on_usable_pred_answerable remains high across checkpoints, suggesting that evidence-copying legality in answerable cases is generally stable. Thus, what the model continues to learn in the second half of training is not chiefly formatting legality, but higher-level task judgment and groundedness quality. Compared with structure-related metrics, the end-to-end metrics show more fluctuation and more selective

differences. e2e_reviewed_correctness_success_rate, e2e_answerability_accuracy, and e2e_semantic_success_rate do not improve monotonically with loss in the mid-to-late stage, but exhibit small oscillations. This indicates that once the model has learned the surface patterns of teacher completions, further decreases in loss do not necessarily translate into continued gains in end-to-end task success.

The thesis ultimately selects step 500 as the best checkpoint. Under all structural hard constraints, it achieves the highest e2e_reviewed_correctness_success_rate, while e2e_answerability_accuracy and e2e_semantic_success_rate are also at or near the best values observed during training. Although later checkpoints continue to show slightly lower loss and maintain similarly strong semantic metrics, they do not exceed

step 500 on the most important user-facing correctness metric. Accordingly, step 500 is taken as the checkpoint that best balances stable protocol reliability and end-to-end task success and is used in the subsequent comparative evaluation on the test set.

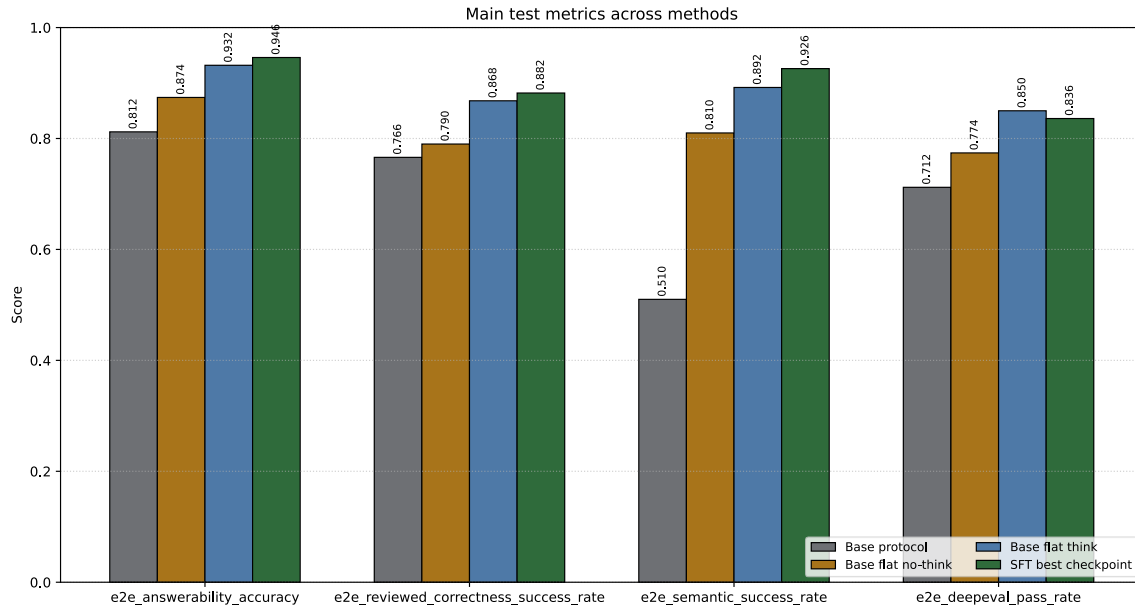
5.2 Main Test Results

On the test set, this thesis compares four formal evaluation pathways: SFT best checkpoint, base protocol, base flat no-think, and base flat think. Among them, base flat no-think is the primary baseline because it corresponds to the default flat QA setting of the base model on the present task and is the direct point of departure that the protocol design and SFT training aim to surpass. base protocol and base flat think serve as auxiliary baselines for clarifying the effects of pure prompt-based protocol constraints and of the base model's think mode, respectively. The main results are shown in Table 9.

Table 9 Main results on the test set

Method	e2e_reviewed_ correctness_ success_rate	e2e_answerabi lity_accuracy	e2e_semantic_ success_rate	e2e_deepeval_ pass_rate
base protocol	0.766	0.812	0.510	0.712
base flat no- think	0.790	0.874	0.810	0.774
base flat think	0.868	0.932	0.892	0.850
SFT best checkpoint	0.882	0.946	0.926	0.836

Figure 4 Main test metrics



After selecting step 500 during validation, this thesis compares it with the three baselines on the test set under the unified evaluation framework. As defined earlier, e2e_reviewed_correctness_success_rate, e2e_answerability_accuracy, e2e_semantic_success_rate, and e2e_deepeval_pass_rate jointly characterize end-to-end success from a user-oriented perspective:

reviewed correctness reflects whether the answer is consistent with the reference answer; answerability accuracy measures whether the model correctly judges when it should answer and when it should refuse;

the semantic and DeepEval metrics jointly evaluate faithfulness of the answer and reasoning process with respect to the given question and evidence, that is, whether they contain semantic content or reasoning steps unsupported by the original evidence.

On the most central metric, e2e_reviewed_correctness_success_rate, SFT best checkpoint achieves the best result (0.882). Relative to the primary baseline base flat no-think, this is an improvement of 9.2 percentage points, corresponding to roughly 43.8% reviewed-correctness error reduction. Relative to base protocol and base flat think, the gains are 11.6 and 1.4 percentage points, respectively. This shows that the proposed method delivers a clear improvement in answer correctness.

On e2e_answerability_accuracy, SFT best checkpoint reaches 0.946. Relative to the primary baseline base flat no-think, the gain is 7.2 percentage points. Relative to the

auxiliary baselines base protocol and base flat think, the gains are 13.4 and 1.4 percentage points, respectively. This indicates that SFT substantially improves the model's ability to identify the boundary between answering and refusing under the given evidence.

For faithfulness, e2e_semantic_success_rate and e2e_deepeval_pass_rate use NLI-based checking and LLM-as-a-judge holistic review, respectively, and are therefore theoretically aligned while complementary in practice. SFT best checkpoint reaches 0.926 on e2e_semantic_success_rate, which is 11.6 percentage points above the primary baseline base flat no-think, and 41.6 and 3.4 percentage points above the auxiliary baselines base protocol and base flat think, respectively.

On e2e_deepeval_pass_rate, it reaches 0.836, exceeding base protocol (0.712) and base flat no-think (0.774) and trailing only slightly behind base flat think (0.850). These results indicate that SFT also effectively suppresses contradictions and unsupported content introduced during reasoning.

5.3 Secondary Analyses

5.3.1 Mechanism Decomposition

The main results reflect end-user usefulness, whereas secondary metrics reveal correctness at different stages and are therefore useful for identifying the concrete weaknesses of each method. We first examine structure- and usability-related metrics, shown in Table 10.

Table 10 Structure- and mechanism-related metrics

Method	usable_rate	parse_ok_rate	protocol_ok_rate_given_parse_ok	evidence_substring_rate_on_usable_pred_answerable
base protocol	0.972	0.986	0.986	0.637
base flat no-think	1.000	N/A	N/A	N/A
base flat think	1.000	N/A	N/A	N/A

SFT best checkpoint	0.998	0.998	1.000	0.990
---------------------	-------	-------	-------	-------

First, usable_rate is very high for all methods, ranging only from 0.972 to 1.000, and is therefore not the main source of differences in the primary results. On the test set, all methods are broadly capable of producing outputs that can be judged further; the key differences arise at the task level rather than the surface-form level.

Comparing base protocol with SFT best checkpoint, the two are already quite close on parse_ok_rate and protocol_ok_rate_given_parse_ok, at 0.986/0.986 versus 0.998/1.000, respectively. In other words, SFT brings only a small improvement over prompt engineering if one looks only at JSON legality. However, on evidence_substring_rate_on_usable_pred_answerable, SFT best checkpoint reaches 0.990, whereas base protocol achieves only 0.637. This indicates that, even under explicit prompt constraints, the base model still tends to rewrite evidence rather than copy it faithfully, whereas SFT greatly strengthens the protocol's core behavior of evidence copying. This confirms the argument made in Section 2.3.3: SFT on qualified demonstrations can effectively teach protocol-execution abilities that are not fully specifiable through prompt text alone.

5.3.2 Answerable vs. Unanswerable

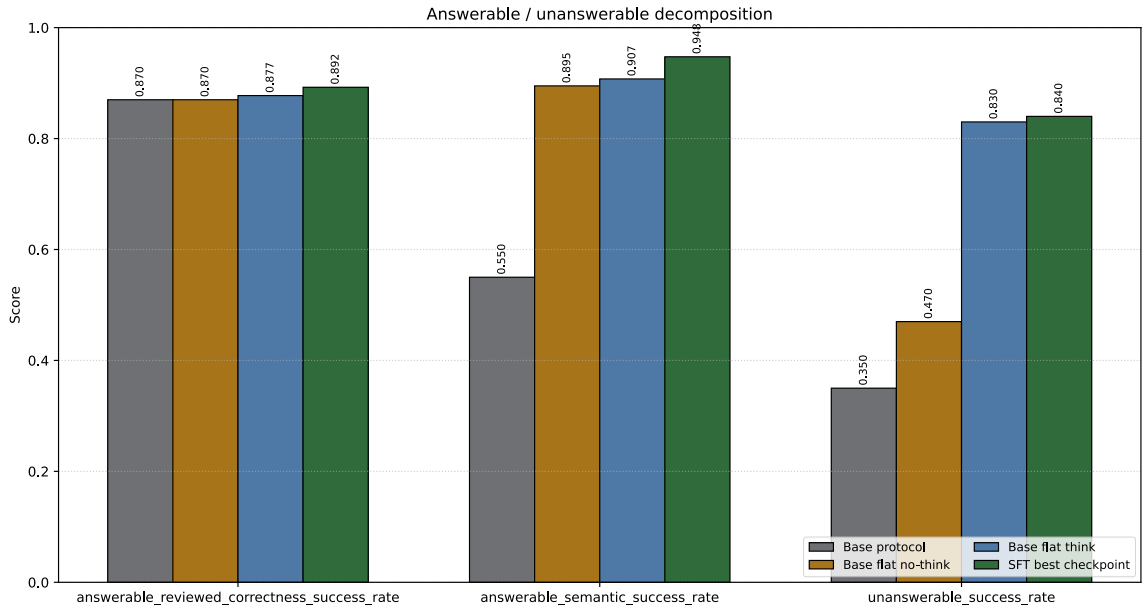
The test data can be further divided into answerable and unanswerable subsets, allowing separate comparison of correctness on answerable and unanswerable samples. The results are shown in Table 11.

Table 11 Answerable / unanswerable stratified results

Method	answerable_reviewed_correctness_success_rate	answerable_semantic_success_rate	unanswerable_success_rate
base protocol	0.8700	0.5500	0.3500
base flat no-think	0.8700	0.8950	0.4700
base flat think	0.8775	0.9075	0.8300

SFT best checkpoint	0.8925	0.9475	0.8400
---------------------	--------	--------	--------

Figure 5 Stratified test metrics



The stratified metrics make the sources of the main-result differences clearer.

On the answerable subset, SFT best

checkpoint achieves 0.8925 on `answerable_reviewed_correctness_success_rate`, higher than the 0.8700 of both base protocol and base flat no-think, and slightly higher than the 0.8775 of base flat think. The

corresponding `answerable_semantic_success_rate` values are 0.9475, 0.5500, 0.8950, and 0.9075, respectively. This shows that SFT brings a modest improvement in answer correctness on answerable samples, but a much larger improvement in semantic faithfulness, that is, in whether reasoning and answering remain faithful to the evidence.

The especially large gain over base protocol indicates that, even under strong prompts and demonstrations, the untuned base model still often produces reasoning and answers not fully supported by the selected evidence, whereas SFT substantially repairs this problem.

On the unanswerable subset, SFT best

checkpoint reaches 0.840 on `unanswerable_success_rate`, while base protocol, base flat no-think, and base flat think achieve 0.350, 0.470, and 0.830, respectively. This indicates that SFT substantially improves the model's ability to recognize when the knowledge is

insufficient to support the question. In the task formulation of this thesis, refusing under knowledge insufficiency is as important as answering correctly when knowledge is sufficient. The result therefore shows that one of the major contributions of SFT to overall performance is its strong suppression of answering behavior that departs from the given knowledge under knowledge-insufficient conditions.

5.3.3 Confidence Effectiveness

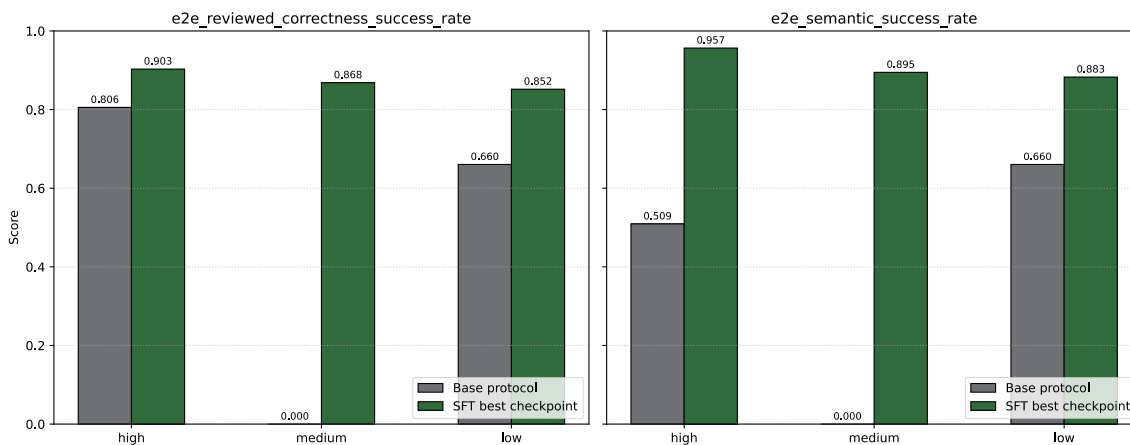
The confidence field exists only for structured methods, so this thesis compares only SFT best checkpoint and base protocol. Table 12 summarizes the confidence-bucket behavior and resolution metrics for the two structured methods.

Table 12 Confidence results for structured methods

Method	bucket	number of samples	reviewed correctness success rate	semantic success rate
base protocol	high	432	0.8056	0.5093
base protocol	medium	1	0.0000	0.0000
base protocol	low	53	0.6604	0.6604
SFT best checkpoint	high	299	0.9030	0.9565
SFT best checkpoint	medium	38	0.8684	0.8947
SFT best checkpoint	low	162	0.8519	0.8827

Figure 6 Confidence-bucket success rates

Confidence bucket success rates



The corresponding normalized resolution values are:

Table 13 Confidence Resolution of structured methods

Method	e2e_confidence_resolution_ reviewed_correctness	e2e_confidence_resolution_semantic
base protocol	0.0199	0.0112
SFT best checkpoint	0.0056	0.0185

From the distributional perspective, the confidence output of base protocol almost collapses into two categories, with high accounting for 88.9% and medium almost absent. By contrast, SFT best checkpoint produces a noticeably more balanced three-way distribution over high / medium / low, suggesting that it has at least learned the discrete self-assessment form required by the protocol rather than simply inheriting the teacher's or base model's overconfidence. Further, in bucketed success rates, SFT exhibits a monotonic pattern of high > medium > low on both reviewed correctness and semantic success, whereas base protocol shows an inverted pattern on semantic success. This indicates that SFT's confidence estimates are at least directionally more meaningful. That said, the discriminative power remains weak. For SFT, the difference between high and low is only about 5.1 percentage points on reviewed correctness success and only about 7.4 points on semantic success, while low-confidence samples still have fairly high success rates. Confidence is therefore not yet strong enough to serve as a robust decision signal. Given that the derived_confidence training target is itself

constructed post hoc by bucketing semantic margins, this outcome is consistent with the nature of its supervision and also reveals a limitation of the current experimental design. Once protocol behavior is stable, additional methods such as DPO or RLHF will likely be needed to improve confidence calibration to the point where it can play a stronger gating role.

5.4 Case Studies

To further explain the concrete behavioral differences behind the aggregate statistics, this thesis conducts a sample-based case analysis using detailed test-set evaluation records.

5.4.1 Success Case: SFT Corrects a Relation-Direction Error on an Answerable Question

We begin with a representative success case. The sample is an answerable question:
Question:

In which city is the ambassador of the Rabat-Sale-Kenitra administrative region to China based?

Knowledge:

Moroccan Ambassador to China: The Moroccan ambassador in Beijing is the official representative of the Government in Rabat to the Government of the People's Republic of China.

Rabat: Rabat is the capital city of Morocco and its second largest city with an urban population of approximately 580,000 (2014) and a metropolitan population of over 1.2 million. It is also the capital city of the Rabat-Sale-Kenitra administrative region.

From this knowledge, it can be inferred strictly that Rabat is the capital of the Rabat-Sale-Kenitra administrative region, and that the phrase "the Government in Rabat" therefore refers to the Moroccan government located in Rabat. At the same time, the knowledge explicitly states "The Moroccan ambassador in Beijing," which means that the ambassador is based in Beijing. The question is therefore answerable, and the correct answer is Beijing.

The structured output produced by SFT best checkpoint is:

Rationale: The knowledge identifies Rabat as the capital of the Rabat-Sale-Kenitra administrative region and states that the Moroccan ambassador representing the Government in Rabat is based in Beijing.

Answer: Beijing

This answer is correct, and its reasoning chain remains within the minimal range required by the protocol. The model first aligns "Government in Rabat" in the first passage with

"Rabat-Sale-Kenitra administrative region" using the second passage, then extracts the ambassador's actual posting city from the first passage, and finally derives the answer.

Table 14 summarizes the outputs of the four methods and their main issues.

Method	Output	Result	Main issue
SFT best checkpoint	Rationale: The knowledge identifies Rabat as the capital of the Rabat-Sale-Kenitra administrative region and states that the Moroccan ambassador representing the Government in Rabat is based in Beijing. Answer: Beijing	Correct	-
Base protocol	Rationale: The evidence states that the Moroccan ambassador in Beijing represents the Government in Rabat, indicating the ambassador is based in Rabat. Answer: Rabat	Incorrect	Reverses the direction of representation; confuses the represented government's location with the ambassador's posting city
Base flat no-think	The question is unclear. The Moroccan ambassador to China is based in Beijing, as stated in the provided knowledge. The Rabat-Sale-Kenitra administrative region is a region in Morocco, and there is no specific mention of an ambassador representing this region to China. Therefore, the answer cannot be determined based on the provided knowledge.	Incorrect	Over-conservative refusal despite sufficient knowledge
Base flat think	I do not know based on the knowledge.	Incorrect	Over-conservative refusal despite sufficient knowledge

The key point of this success case is that SFT best checkpoint corrects a representative error pattern: misunderstanding the direction of a relation. base protocol has already identified the right evidence, but misreads "the ambassador in Beijing represents the Government in Rabat" as "the ambassador is based in Rabat," conflating the location of the represented government with the diplomat's posting city. The two flat baselines are more conservative and refuse outright, misclassifying a question that is in fact strictly answerable from the knowledge as knowledge-insufficient. By contrast, SFT not only preserves evidence grounding but also more stably resolves the relation and direction demanded by the question. This suggests that the gains of the proposed method are not limited to more stable formatting or more reliable evidence copying, but arise from coordinated error reduction across answerability judgment, relation-semantic alignment, and minimal reasoning execution.

5.4.2 Failure Case I: Answering an Unanswerable League Question from Incomplete Team Information

The first failure pattern is that the model misclassifies an unanswerable sample as answerable and proceeds to answer. The sample is:

Question:

What league was the team that played home games at Holman Stadium belong to?

Its corresponding knowledge is:

Knowledge:

Pittsfield Colonials: The team was previously known as the American Defenders of New Hampshire (often simply referred to as the "Defenders"), and prior to that was known as the Nashua Pride.

Nashua Pride: The Nashua Pride were a professional baseball team based in Nashua, New Hampshire, in the United States, not affiliated with Major League Baseball. They played home games at Holman Stadium from 1998 through 2008, when they were sold and renamed the American Defenders of New Hampshire. In 2010 that team moved to Pittsfield, Massachusetts, and became the Pittsfield Colonials.

This knowledge clearly identifies the relevant team as the *Nashua Pride* and tells us that it was *a professional baseball team and not affiliated with Major League Baseball*.

However, it does not provide the name of the league to which the team belonged.

Therefore, under the task definition of this thesis, the question should be judged unanswerable: the current knowledge is insufficient to support a verifiable short answer.

Nevertheless, SFT best checkpoint produces:

Rationale: The knowledge states that the Nashua Pride played home games at Holman Stadium and explicitly defines them as a professional baseball team not affiliated with Major League Baseball.

Answer: not affiliated with Major League Baseball

The failure here is that the model correctly identifies the team referred to by the question and locates relevant local information about that team, but mistakenly treats "not affiliated with Major League Baseball" as if it were the required league description. The error does not occur at the evidence-selection level, but at the level of question-slot alignment: the question asks for a league name, while the model fills that slot with an affiliation status.

This failure is representative because it is not a complete departure from the knowledge, but rather a case of unsupported slot substitution. The model does extract real spans from the knowledge, yet it misunderstands what type of information the question requires.

Superficially, the output remains grounded; in task terms, however, it fails to answer the field actually requested. The case shows that even after SFT, on some boundary samples where the knowledge is locally rich but globally insufficient, the model may still treat

nearby but non-equivalent information as a sufficient answer and therefore cross the refusal boundary.

5.4.3 Failure Case II: Entering the Answerable Path but Drifting Away from the Question's Semantic Criterion

The second failure pattern is that the model correctly enters the answerable path and even produces locally supported reasoning, but the final answer is still wrong at the task level.

A clear example is:

Question:

Are both American Zeitgeist and Michael Jackson's This Is It documentary films about entertainment?

Its corresponding knowledge is:

Knowledge:

American Zeitgeist: American Zeitgeist is a 2006 documentary film by Rob McGann. It discusses the war on terror and religion. It was the winner to the best feature-length documentary at the Houston International Film Festival.

Michael Jackson's This Is It: Michael Jackson's This Is It is a 2009 American documentary-concert film directed by Kenny Ortega that documents Michael Jackson's rehearsals and preparation for his concert series of the same name that was originally scheduled to start on July 13, 2009, but was cancelled due to his death eighteen days prior on June 25. The film consists of Jackson rehearsing musical numbers, directing his team, and additional behind-the-scenes footage including dancer auditions and costume design.

From this knowledge, one can directly judge that the latter is clearly entertainment-related, whereas the former is about *the war on terror and religion* and is not about entertainment. The question is therefore answerable, and the correct answer should be no.

The output of SFT best checkpoint is:

Rationale: The provided text explicitly identifies 'American Zeitgeist' as a documentary film and describes 'Michael Jackson's This Is It' as a documentary-concert film, confirming both are documentary films.

Answer: yes

The crucial failure here is that the model rewrites the original question into an easier but different one. The actual question is whether both films are *about entertainment*; the model's rationale instead answers whether both belong to the category of *documentary films*. In other words, the model exhibits clear question drift: it preserves the surface form of the question, but silently changes the decision criterion from thematic content to genre category.

This case indicates that the current method, while already constraining evidence fidelity and structured output reasonably well, still does not control question-to-rationale alignment sufficiently. The model cites true and supportable content, but performs a reasoning task that does not match the original question. This is a more subtle kind of failure than conventional hallucination: the local semantics are supported, yet the answer is off-target at the task level.

5.4.4 Summary of the Case Studies

The case studies show that protocol-based SFT training has already brought substantial progress on the core capabilities targeted in this thesis: locating key relevant evidence and carrying out constrained minimal reasoning over that evidence so as to produce more grounded answers.

At the same time, the cases reveal the main remaining weaknesses and room for improvement. On more complex samples, the model still struggles with recognizing the true object of the question and correctly aligning the objects, relations, or slot requirements in the question with their corresponding expressions in the knowledge. The current method improves evidence finding and evidence-grounded answering, but the higher-level capability of question-knowledge alignment still needs to be strengthened.

5.5 Summary and Discussion

Overall, the proposed protocol-based SFT method achieves stable and practically meaningful gains on the test set. Relative to the primary baseline base flat no-think, SFT best checkpoint improves the most central metric, `e2e_reviewed_correctness_success_rate`, by 9.2 percentage points, `e2e_answerability_accuracy` by 7.2 points, and `e2e_semantic_success_rate` by 11.6 points. This means that, compared with the base model's default QA setting, SFT not only improves final answer correctness but also reduces unsupported reasoning that drifts away from the provided context during intermediate reasoning. Relative to the auxiliary baselines base protocol and base flat think, the results further show that a protocol internalized through SFT substantially outperforms prompt-only and example-only constraints, and also maintains a modest advantage over generic reasoning on grounded question answering.

The secondary analyses further clarify where these overall gains come from. The benefits of SFT do not arise mainly from improvements in surface-form legality. `usable_rate`, `parse_ok_rate`, and `protocol_ok_rate_given_parse_ok` are already close to saturation for both base protocol and SFT best checkpoint, which suggests that prompt engineering alone is already sufficient for the base model to generate outputs that are formally legal and parseable. What SFT strengthens is strict evidence copying, a reduction in answerable cases where evidence appears relevant but does not truly support the reasoning, and a significant improvement in refusal under knowledge insufficiency. This supports the thesis's central claim: for the protocol-constrained grounded QA task studied here, the difficult and outcome-critical component is not surface formatting, but rather a set of implicit execution rules that are hard to specify exhaustively in natural-language prompts yet crucial for final behavior, such as judging information sufficiency, extracting key evidence, and reasoning strictly from selected evidence. Prompt engineering can hint at these requirements to some extent, but cannot guarantee that the model has internalized them as stable generation habits. Under high-quality demonstrations, SFT turns these implicit capabilities into reliably callable behavior patterns.

Comparison with the two flat baselines also shows that the gains of the proposed method are not equivalent to simply encouraging the model to "think more." base flat think is already a strong auxiliary baseline, with high values on `e2e_reviewed_correctness_success_rate`, `e2e_answerability_accuracy`, and `e2e_deepeval_pass_rate`, indicating that the base model's think mode does substantially improve overall answer quality. Nevertheless, SFT best checkpoint still retains an advantage on multiple core metrics, suggesting that the main benefit of the proposed method lies in protocolized reasoning, that is, task-specific reasoning constrained by evidence boundaries rather than freely expanded general reasoning. Even so, the experiments also reveal several limitations.

First, the confidence analysis shows that although the model learns a more balanced confidence distribution than base protocol and develops a weak but correctly ordered discrimination signal, this signal is still too weak to function as a strong decision basis.

This indicates that the current mechanism for constructing `derived_confidence` remains limited.

Second, the case studies show that beyond evidence extraction and reasoning, a key difficulty in grounded QA lies in complex question-object recognition and in aligning question objects with expressions in the knowledge. On many samples, the model can already stably locate relevant evidence and answer from it. On complex or boundary samples, however, it can still mistake related but non-equivalent knowledge for sufficient evidence because it fails to fully recognize the required object type, slot requirement, or decision criterion.

5.6 Threats to Validity

5.6.1 Internal validity

Several aspects of the evaluation pipeline may affect whether the observed gains can be attributed solely to protocol-based SFT.

Protocol outputs are evaluated directly from their explicit structured fields, whereas flat-output baselines require an auxiliary LLM to extract a comparable short answer and detect refusal before answerability and correctness can be scored. Although this design is necessary to place protocol and non-protocol outputs under a unified evaluation path, it introduces an additional model-mediated step for flat baselines and may therefore create evaluation asymmetry.

Some components of the overall pipeline, such as teacher prompting, supervision filtering, and checkpoint selection, are tightly coupled, so the final gains should be interpreted as the effect of the proposed end-to-end workflow rather than of a single isolated design choice.

5.6.2 Construct validity

The thesis operationalizes trustworthiness in this setting primarily as faithfulness to the provided knowledge, rather than open-domain factual correctness. The restriction is deliberate and aligns with the task formulation, but it also means that the reported results should be interpreted within this bounded-evidence definition only.

Moreover, key evaluation QA signals such as semantic support, answer equivalence, and hallucination are approximated through automatic checkers, including NLI-based

judgment, answer-equivalence matching, rule-based QA-consistency checks, and DeepEval as an LLM-as-judge signal. While these checkers are calibrated where possible and combined to reduce reliance on any single metric, none of them is a perfect proxy for grounded reasoning quality, and residual mismatch between the intended construct and the implemented metrics may remain.

5.6.3 External validity

The main experiments are conducted on the HotpotQA distractor subset and focus on Qwen3-4B as the student model. Although this setting is appropriate for controlled study of grounded multi-hop short-answer QA, it does not cover other data regimes such as long-document QA, naturally occurring retrieval failures, domain-specific enterprise corpora, or other model families and scales. In particular, the unanswerable cases in this thesis are derived from originally answerable HotpotQA samples by minimally breaking the support chain. This design improves controllability, but such constructed negatives may not fully capture the diversity and difficulty of naturally occurring unanswerable questions in real-world retrieval-augmented settings. Therefore, the results should be viewed as evidence for the effectiveness of the proposed approach in a controlled grounded-QA setting, rather than as a universal guarantee across tasks and deployment scenarios.

5.6.4 Statistical conclusion validity

The reported results are obtained on fixed train/validation/test partitions and are based primarily on single-run training and evaluation under deterministic decoding. The thesis does not provide multi-seed training variance, confidence intervals, or formal significance tests. As a result, while the observed gains are substantial at the reported test-set level, the robustness of these gains across random seeds, alternative splits, and repeated runs remains to be established in future work.

6 Conclusion and Future Work

6.1 Conclusion

This thesis investigates reliable question answering with large language models under a given-knowledge setting and formulates hallucination mitigation as the problem of ensuring faithfulness to the provided knowledge. Specifically, when the knowledge is sufficient to support an answer, the model should answer on that basis; when the knowledge is insufficient to support a verifiable answer, the model should refuse rather than guess. Under this formulation, the thesis models QA as a constrained structured decision process consisting of answerability judgment, evidence extraction, minimal reasoning, answer generation, and confidence self-assessment, and proposes a corresponding protocolized output format.

Methodologically, the thesis develops a relatively complete technical pipeline around this protocol. First, it constructs answerable and derived unanswerable samples from the HotpotQA distractor subset, thereby forming training and evaluation data that cover both answerable cases and knowledge-insufficient cases. Second, it uses a strong teacher model to generate candidate structured supervision and filters and calibrates these samples with parse, protocol, consistency, evidence-substring, correctness, and semantic checkers. Finally, it performs protocol-based SFT on Qwen3-4B using LoRA and compares the resulting model in a unified evaluation framework against the primary baseline base flat no-think and two auxiliary baselines, base protocol and base flat think. The experimental results show that protocol-based supervised fine-tuning brings stable and practically meaningful overall gains on the test set. Relative to the primary baseline base flat no-think, the best SFT model reaches 88.2%, 94.6%, and 92.6% on end-to-end answer correctness, answerability accuracy, and semantic faithfulness, respectively, corresponding to improvements of 9.2, 7.2, and 11.6 percentage points. These improvements arise because the model's answering behavior under evidence constraints is substantially improved, specifically through more accurate refusal-boundary control, more reliable evidence citation, and minimal reasoning that remains more faithful to the given knowledge.

Overall, the results indicate that, for QA grounded in given knowledge, relying only on prompt engineering or on a generic thinking mode is insufficient to stably obtain constrained and verifiable answering behavior. Explicitly encoding the target behavior into a protocol and performing SFT with high-quality demonstrations is a more effective way to internalize the behavior pattern of judging first, gathering evidence second, and answering last into a relatively small model.

6.2 Main Contributions

The main contributions of this thesis are as follows.

First, starting from the problem of faithfulness under given knowledge, the thesis proposes a structured output protocol for grounded QA. The protocol decomposes the previously implicit and mixed generation process into five interdependent subtasks: answerability, evidence, rationale, answer, and confidence, thereby making the model output easier to supervise, verify, and calibrate.

Second, the thesis builds a complete methodological framework around this protocol, covering data construction, teacher-supervision generation, training, and evaluation. This includes derived-unanswerable sample construction, multi-stage rule-based and model-based checkers, and calibration procedures for NLI and answer-equivalence checkers based on human annotation. The contribution is therefore not merely an output format, but a reproducible experimental paradigm for protocol-based SFT.

Third, through systematic comparison with multiple baselines, the thesis empirically shows that protocol-based SFT yields stable gains in end-to-end correctness, answerability judgment, and semantic faithfulness, and further reveals that its main mechanism-level benefits come from stronger evidence extraction, better refusal control, and improved relation-semantic alignment.

6.3 Future Work

Although the proposed method achieves clear improvements, it still has several limitations. First, the case analyses show that the current model still suffers from insufficient question-knowledge alignment on some complex samples. In such cases, the model may identify real and locally supported evidence, yet still fail to answer the actual object, relation, or slot requested by the question. This suggests that the method already

improves evidence faithfulness effectively, but still needs stronger high-level semantic parsing of questions and better slot alignment.

Second, although the confidence field begins to show a directionally meaningful bucket pattern, its discriminative power remains weak and is still insufficient to serve as a strong gating signal. Future work may build on stable protocol behavior and introduce DPO, RLHF, or specialized calibration objectives to further improve the interpretability and decision value of confidence.

Third, the experiments in this thesis are conducted mainly on data derived from HotpotQA, using Qwen3-4B and a given-knowledge setting. The external validity of the conclusions therefore remains to be tested on more datasets, more model scales, and more task types. Future work can investigate the applicability and robustness of the protocol across domain-specific document QA, different language settings, and longer-context conditions.

Finally, this thesis deliberately limits its scope to the provided-knowledge setting and does not jointly model open-system factors such as retrieval errors or insufficient knowledge recall. Future work may extend the protocolized approach proposed here to retrieval-augmented generation and develop a more complete hierarchical reliability-control mechanism spanning retrieval, evidence selection, answering, and refusal.

Appendices

Appendix A Prompt Templates

Appendix A.1 Teacher Prompt Template

```
You are preparing structured grounded-QA supervision data.
Return exactly one JSON object and nothing else.
Requirements:
1) Use the field order: answerability, evidence, rationale, answer,
confidence.
2) Every evidence quote must be copied from the provided knowledge.
3) Evidence must be the smallest sufficient span or spans from the knowledge
needed to answer the question.
4) The rationale must use only the information contained in the evidence.
5) The rationale must perform only the minimal reasoning needed to reach the
answer.
6) The rationale must not be identical to the final answer text.
7) When the question is answerable, cite evidence and answer directly.
8) When the question is unanswerable, use an allowed refusal template and keep
evidence as [].
9) For unanswerable cases, the rationale must briefly identify what
information is missing, ambiguous, or contradictory in the provided knowledge.
10) Preserve the natural answer form when possible.
Reference answer:
[REFERENCE_ANSWER]
Keep your final answer semantically consistent with the reference answer.
JSON schema:
[SCHEMA_JSON]
Allowed refusal templates:
[
  "I don't know based on the provided knowledge.",
  "The provided knowledge does not contain enough information to answer the
question.",
  "I cannot answer from the provided knowledge."
]
Knowledge:
[KNOWLEDGE]
Question: [QUESTION]
```

Appendix A.2 Teacher Prompt + Few-Shot Template

The template below corresponds to the default teacher_fewshot version used by the formal base protocol baseline, with fewshot_k=2.

Few-shot format examples (follow the same JSON schema and field order):

```
Example 1
Knowledge:
DocA: The capital of Italy is Rome.
Question: What is the capital of Italy?
Output:
```



```

{
  "answerability": "answerable",
  "evidence": [
    {
      "quote": "The capital of Italy is Rome."
    }
  ],
  "rationale": "The evidence directly states the capital of Italy.",
  "answer": "Rome",
  "confidence": "high"
}

```

Example 2

Knowledge:

DocB: Marie Curie discovered polonium.

DocC: The document does not mention her birthplace.

Question: Where was Marie Curie born?

Output:

```

{
  "answerability": "unanswerable",
  "evidence": [],
  "rationale": "The provided knowledge mentions discoveries but does not provide her birthplace.",
  "answer": "I don't know based on the provided knowledge.",
  "confidence": "low"
}

```

Now solve the next case.

You are preparing structured grounded-QA supervision data.

Return exactly one JSON object and nothing else.

Requirements:

- 1) Use the field order: answerability, evidence, rationale, answer, confidence.
- 2) Every evidence quote must be copied from the provided knowledge.
- 3) Evidence must be the smallest sufficient span or spans from the knowledge needed to answer the question.
- 4) The rationale must use only the information contained in the evidence.
- 5) The rationale must perform only the minimal reasoning needed to reach the answer.
- 6) The rationale must not be identical to the final answer text.
- 7) When the question is answerable, cite evidence and answer directly.
- 8) When the question is unanswerable, use an allowed refusal template and keep evidence as [].
- 9) For unanswerable cases, the rationale must briefly identify what information is missing, ambiguous, or contradictory in the provided knowledge.
- 10) Preserve the natural answer form when possible.

Reference answer:

[REFERENCE_ANSWER]

Keep your final answer semantically consistent with the reference answer.

JSON schema:

[SCHEMA_JSON]

Allowed refusal templates:

```

[
  "I don't know based on the provided knowledge.",
  "The provided knowledge does not contain enough information to answer the question.",
  "I cannot answer from the provided knowledge."
]

```

```
]
Knowledge:
[KNOWLEDGE]
Question: [QUESTION]
```

Appendix A.3 Infer Prompt Template

```
Return exactly one JSON object with fields in this order:
['answerability', 'evidence', 'rationale', 'answer', 'confidence']
Rules:
- If answerable, cite 1 to 4 quotes copied from the knowledge.
- Provide a short rationale showing how the evidence supports the answer.
- The rationale must not be identical to the answer.
- If unanswerable, set evidence to [] and use an allowed refusal template.
- In unanswerable cases, the rationale should briefly state what key
information is missing, ambiguous, or contradictory.
- Keep the answer concise and directly responsive.
- Do not add explanations outside the JSON.
Refusal templates: I don't know based on the provided knowledge.
Knowledge:
[KNOWLEDGE]
Question: [QUESTION]
```

Appendix A.4 Plain Knowledge QA Prompt (Refusal-Enabled)

The template below corresponds to the actual prompt used by the flat baselines

```
Use only the provided knowledge to answer the question.
If the knowledge is insufficient, answer exactly: I do not know based on the
knowledge.
Do not repeat or paraphrase this instruction in the answer.
[KNOWLEDGE]
Question: [QUESTION]
Answer:
```

Appendix B Core Implementation Excerpts

Appendix B.1 Token-Budget Check

```
prompt = build_infer_prompt(
    knowledge=str(knowledge or "").strip(),
    question=str(question or "").strip(),
)
prompt_tokens = count_text_tokens(prompt, tokenizer_name)
keep = prompt_tokens <= max_prompt_tokens
```

Appendix B.2 EM / Token-F1 Correctness Check

```
exact_match = _normalize_answer(answer) == _normalize_answer(reference_answer)
token_f1 = _token_f1(answer, reference_answer)
semantic_match = bool(exact_match or token_f1 >=
resolved_cfg.semantic_match_f1_threshold)
```

Appendix B.3 Structured Semantic Hypothesis Construction

```
def _build_semantic_hypothesis(output: StructuredQaOutput) -> str:
    rationale = str(output.rationale or "").strip()
    answer = str(output.answer or "").strip()
    return f"{rationale}\nTherefore the answer is {answer}"
```

Appendix B.4 Temperature Scaling and Margin Decision for NLI

```
def _temperature_calibrated(self, sc: NLIScores) -> Dict[str, float]:
    logp_e = math.log(max(sc.entail, eps))
    logp_n = math.log(max(sc.neutral, eps))
    logp_c = math.log(max(sc.contradict, eps))
    scaled = [logp_e / self.cfg.temperature,
              logp_n / self.cfg.temperature,
              logp_c / self.cfg.temperature]
    p_e, p_n, p_c = self._softmax(scaled)
    margin = scaled[0] - max(scaled[1], scaled[2])
    return {
        "entail": float(p_e),
        "neutral": float(p_n),
        "contradict": float(p_c),
        "margin": float(margin),
    }
def _nli_decision_full(self, margin: float) -> bool:
    return bool(margin >= self.cfg.full_margin_threshold)
```

Appendix B.5 Answer Equivalence Matcher Review

```
score = float(self.matcher.get_score(reference_answer, candidate_answer,
question))
match = bool(score >= float(self.cfg.threshold))
```

References

- [1] T. Brown *et al.*, “Language Models Are Few-Shot Learners,” *ArXiv (Cornell University)*, vol. 4, May 2020, doi: <https://doi.org/10.48550/arxiv.2005.14165>.
- [2] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” *arXiv (Cornell University)*, Mar. 2022, doi: <https://doi.org/10.48550/arxiv.2203.02155>.
- [3] Z. Ji *et al.*, “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, vol. 55, no. 12, Nov. 2022, doi: <https://doi.org/10.1145/3571730>.
- [4] L. Huang *et al.*, “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions,” *ACM transactions on office information systems*, vol. 43, no. 2, Nov. 2024, doi: <https://doi.org/10.1145/3703155>.
- [5] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, “On Faithfulness and Factuality in Abstractive Summarization,” *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, doi: <https://doi.org/10.18653/v1/2020.acl-main.173>.
- [6] O. Honovich *et al.*, “TRUE: Re-evaluating Factual Consistency Evaluation,” *arXiv.org*, 2022. <https://arxiv.org/abs/2204.04991>
- [7] W. Kryściński, B. McCann, C. Xiong, and R. Socher, “Evaluating the Factual Consistency of Abstractive Text Summarization,” *arXiv (Cornell University)*, Oct. 2019, doi: <https://doi.org/10.48550/arxiv.1910.12840>.
- [8] A. Wang, K. Cho, and M. Lewis, “Asking and Answering Questions to Evaluate the Factual Consistency of Summaries,” *arXiv (Cornell University)*, Apr. 2020, doi: <https://doi.org/10.48550/arxiv.2004.04228>.
- [9] E. Durmus, H. He, and M. Diab, “FEQA: A Question Answering Evaluation Framework for Faithfulness Assessment in Abstractive Summarization,” *ACLWeb*, Jul. 01, 2020. <https://aclanthology.org/2020.acl-main.454/>
- [10] A. R. Fabbri, C.-S. Wu, W. Liu, and C. Xiong, “QAFactEval: Improved QA-Based Factual Consistency Evaluation for Summarization,” *arXiv (Cornell University)*, Dec. 2021, doi: <https://doi.org/10.48550/arxiv.2112.08542>.

- [11] J. Z. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *arXiv*, Jan. 2022, doi: <https://doi.org/10.48550/arxiv.2201.11903>.
- [12] Z. Yang *et al.*, “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering,” Sep. 2018, doi: <https://doi.org/10.18653/v1/d18-1259>.
- [13] A. P. Parikh, O. Täckström, D. Das, and J. Uszkoreit, “A Decomposable Attention Model for Natural Language Inference,” *arXiv (Cornell University)*, Jun. 2016, doi: <https://doi.org/10.48550/arxiv.1606.01933>.
- [14] Z. Li, I. Mondal, Y. Liang, H. Nghiem, and J. L. Boyd-Graber, “PEDANTS: Cheap but Effective and Interpretable Answer Equivalence,” *arXiv.org*, 2024. <https://arxiv.org/abs/2402.11161>.
- [15] C. Si, C. Zhao, and J. Boyd-Graber, “What’s in a Name? Answer Equivalence For Open-Domain Question Answering,” *arXiv.org*, 2021. <https://arxiv.org/abs/2109.05289>.
- [16] “Hallucination | DeepEval by Confident AI - The LLM Evaluation Framework,” *Deepeval.com*, Mar. 17, 2026. <https://deepeval.com/docs/metrics-hallucination> (accessed Mar. 21, 2026).
- [17] “Supervised Fine-tuning Trainer,” *huggingface.co*. https://huggingface.co/docs/trl/sft_trainer
- [18] E. J. Hu *et al.*, “LoRA: Low-rank adaptation of large language models,” *arXiv (Cornell University)*, Jan. 2021, doi: <https://doi.org/10.48550/arxiv.2106.09685>.
- [19] A. Yang *et al.*, “Qwen3 Technical Report,” *arXiv (Cornell University)*, May 2025, doi: <https://doi.org/10.48550/arxiv.2505.09388>.
- [20] A. H. Murphy, “A New Vector Partition of the Probability Score,” *Journal of Applied Meteorology (1962/1982)*, vol. 12, no. 4, pp. 595–600, 1973, doi: <https://doi.org/10.2307/26176769>.
- [21] M. Laurer, Wouter van Atteveldt, A. Casas, and Kasper Welbers, “Less Annotating, More Classifying: Addressing the Data Scarcity Issue of Supervised Machine Learning with Deep Transfer Learning and BERT-NLI,” pp. 1–33, Jun. 2023, doi: <https://doi.org/10.1017/pan.2023.20>.